

OPTICAL MUSIC RECOGNITION FOR JAZZ LEAD SHEETS

A CRNN–CTC Approach for Monophonic Lines

Pol Casanovas Puig

Supervisor: Manel Frigola Bourlon

Department: Automatic Control (ESAI) – FIB/UPC

Degree: Bachelor's in Informatics Engineering (Computing)

Speciality: Computing

Facultat d'Informàtica de Barcelona (FIB)
Universitat Politècnica de Catalunya (UPC) – BarcelonaTech

Spring 2026

Resum

El *Reconeixement Òptic de Música* (OMR) és el procés d'obtenir representacions simbòliques llegibles per màquina a partir d'imatges de partitures. Malgrat els avenços en el camp, la transcripció automàtica de partitures reals de baixa qualitat continua essent un repte obert. Aquesta memòria presenta un sistema OMR orientat a línies de melodia monofòniques de jazz del recull *The Real Book*.

El sistema adopta una arquitectura de dos fluxos: un model *CRNN-CTC* per al reconeixement de la línia melòdica i un segon *CRNN-CTC* per a l'extracció dels símbols d'acords. Tots dos models s'entrenen de manera independent sobre dades sintètiques generades a partir del conjunt PrIMuS. Per reduir la diferència entre imatges sintètiques i escaneigs reals, s'aplica una etapa d'augmentació que simula artefactes fotogràfics habituals. Avaluat amb la *Taxa d'Error de Símbols* (SER), el model assoleix un **1,19%** sobre el conjunt de prova sintètic net i un **1,28%** sobre la versió simulada com a escanejada; la SER melòdica (excloent-hi els tokens de compàs i lligadura) és de **0,11%** i **0,18%** respectivament, i el **73%** i **71%** de les transcripcions són perfectes per a cada versió.

El sistema complet integra detecció de pentagrames, reconeixement de melodia, extracció de símbols d'acords i correcció gramatical, i accepta fitxers PDF o imatges com a entrada. La sortida és un document JSON estructurat que inclou la seqüència de tokens LMX i els acords per a cada pentagrama detectat. Aquest treball demostra la viabilitat dels models *CRNN-CTC* per al domini específic de les línies de jazz monofòniques i estableix les bases per a extensions futures cap a la transcripció polifònica.

Resumen

El *Reconocimiento Óptico de Música* (OMR) es el proceso de obtener representaciones simbólicas legibles por máquina a partir de imágenes de partituras. A pesar de los avances en el campo, la transcripción automática de partituras reales de baja calidad sigue siendo un desafío abierto. Esta memoria presenta un sistema OMR orientado a líneas de melodía monofónicas de jazz de la colección *The Real Book*.

El sistema adopta una arquitectura de dos flujos: un modelo *CRNN-CTC* para el reconocimiento de la línea melódica y un segundo *CRNN-CTC* para la extracción de los símbolos de acorde. Ambos modelos se entrenan de forma independiente sobre datos sintéticos generados a partir del conjunto PrIMuS. Para reducir la brecha entre imágenes sintéticas y escáneres reales, se aplica una etapa de aumento que simula artefactos fotográficos habituales. Evaluado con la *Tasa de Error de Símbolos* (SER), el modelo alcanza un **1,19%** sobre el conjunto de prueba sintético limpio y un **1,28%** sobre la versión simulada como escaneada; la SER melódica (excluyendo tokens de compás y ligadura) es de **0,11%** y **0,18%** respectivamente, y el **73%** y **71%** de las transcripciones son perfectas para cada versión.

El sistema completo integra detección de pentagramas, reconocimiento de melodía, extracción de símbolos de acorde y corrección gramatical, y acepta archivos PDF o imágenes como entrada. La salida es un documento JSON estructurado que incluye la secuencia de tokens LMX y los acordes para cada pentagrama detectado. Este trabajo demuestra la viabilidad de los modelos *CRNN-CTC* para el dominio específico de las líneas de jazz monofónicas y sienta las bases para extensiones futuras hacia la transcripción polifónica.

Abstract

Optical Music Recognition (OMR) is the task of automatically converting images of musical scores into machine-readable symbolic representations. Despite advances in the field, transcribing real-world scans of informal or low-quality sources remains an open challenge. This thesis presents an end-to-end OMR system targeting monophonic melody lines in jazz lead sheets, with a focus on the *Real Book* collection.

The system follows a two-stream architecture: a *Convolutional Recurrent Neural Network with Connectionist Temporal Classification* loss (CRNN-CTC) for melody recognition, and a second CRNN-CTC for chord symbol extraction. Both streams are trained independently on synthetic data derived from the PrIMuS dataset. To bridge the gap between clean synthetic renderings and real scans, an augmentation stage simulates common photographic artefacts. Evaluated using the *Symbol Error Rate* (SER) on a held-out test split of 4 604 samples, the melody model achieves **1.19%** SER on the clean synthetic set and **1.28%** on its scan-simulated counterpart, with melodic SER (barline and tie tokens excluded) of **0.11%** and **0.18%** respectively, and perfect transcription rates of **73%** and **71%**.

The full inference pipeline integrates staff detection, melody recognition, chord symbol extraction, and a grammar correction stage, accepting PDF or image input and returning a structured JSON document with LMX token sequences and chord annotations for each detected staff. This work demonstrates the feasibility of CRNN-CTC-based OMR for the specific domain of monophonic jazz lead sheets and lays the groundwork for future extensions towards polyphonic transcription.

Contents

Resum (Català)	i
Resumen (Español)	ii
Abstract (English)	iii
1 Introduction and Context	1
1.1 Motivation	1
1.1.1 The Digitization Gap in Music	1
1.1.2 Jazz Lead Sheets and <i>The Real Book</i>	2
1.1.3 Why Existing OMR Tools Fall Short	2
1.2 Musical Notation Fundamentals	2
1.2.1 The Staff and the Treble Clef	3
1.2.2 Pitch, Accidentals, and Key Signatures	3
1.2.3 Note Duration and Rhythm	4
1.2.4 Time Signatures and Measures	4
1.2.5 Chord Symbols	5
1.2.6 The Lead-Sheet Format	5
1.3 Problem Formulation	5
1.4 Objectives	7
1.5 Proposed Approach	7
1.6 Scope	8
1.7 Document Structure	9
2 State of the Art	10
2.1 The Classical OMR Pipeline	10
2.2 Deep Learning Paradigm: CRNN with CTC	12
2.3 End-to-End Transformer Architectures	12
2.4 Datasets for OMR Training	13
2.5 Symbolic Output Representations	13
2.6 Evaluation Metrics	15
2.7 OMR for Jazz Lead Sheets	15
2.8 Summary	15
3 Project Management (GEP)	17
3.1 Justification of the Chosen Architecture	17
3.1.1 Existing alternatives	17
3.1.2 Selection criteria	17
3.1.3 Chosen alternative	18
3.2 Methodology	19
3.3 Project Planning	20
3.3.1 Work-breakdown structure	20

3.3.2	Tasks, dependencies, and resources	21
3.3.3	Gantt chart	21
3.4	Risk Register and Contingency Plans	24
3.5	Economic Management	25
3.5.1	Budget structure	25
3.5.2	Human resources	25
3.5.3	General costs, contingency, and incidentals	26
3.5.4	Total budget	27
3.5.5	Budget control	27
4	Design	29
4.1	Problem Decomposition	29
4.2	Data Strategy	30
4.2.1	Synthetic supervision from PrIMuS	30
4.2.2	Domain augmentation: from clean renders to scanned pages	30
4.2.3	Domain restriction: filtering the training set	31
4.2.4	Fine-tuning on real pages	31
4.3	Output Representation: LMX	31
4.3.1	Grammar	31
4.3.2	Vocabulary	32
4.3.3	Why LMX over alternatives	32
4.4	Model Design	32
4.4.1	Why CRNN-CTC	32
4.4.2	Backbone choice	33
4.4.3	Temporal resolution and width compression	33
4.4.4	Recurrent layer	33
4.5	Two-Stream Architecture for Lead Sheets	34
4.5.1	Why two networks rather than one	34
4.5.2	Independent training, joint inference	34
4.5.3	Chord vocabulary and notation conventions	34
4.6	Training Strategy	34
4.6.1	Two-phase plan	34
4.6.2	Loss and supervision signal	35
4.6.3	Data splits	35
4.6.4	Header-stripping augmentation	35
4.6.5	Rare-token oversampling	35
4.7	Grammar Correction as a Design Layer	35
4.8	Evaluation Design	36
4.8.1	Primary metric: Symbol Error Rate	36
4.8.2	Category-level error breakdown	36
4.8.3	Melodic SER	36
4.8.4	Domain-gap evaluation	36
5	Implementation	37
5.1	Technology Stack	37
5.2	Repository Structure	38
5.3	Data Pipeline Implementation	38
5.3.1	Render: PrIMuS to LilyJAZZ	38
5.3.2	Convert: PrIMuS .semantic to LMX	39
5.3.3	Augment: scan simulation	39
5.3.4	Vocab: build the token index	40
5.4	Model Implementation	40

5.4.1	Module structure	40
5.4.2	ResNet18 backbone modifications	41
5.4.3	Bidirectional LSTM	41
5.4.4	CTC loss configuration	41
5.4.5	Training loop	41
5.4.6	Hyperparameter summary	42
5.5	Chord OCR Implementation	42
5.5.1	Synthetic chord-strip generation	42
5.5.2	Chord network architecture	43
5.5.3	Fine-tuning on hand-labelled strips	43
5.6	Inference Pipeline Implementation	43
5.6.1	Preprocessing	43
5.6.2	Staff detection	43
5.6.3	Music recognition	44
5.6.4	Chord recognition	45
5.6.5	Grammar fixer	45
5.7	Command-Line Interface	45
5.8	Web Service	46
5.9	Reproducibility	46
6	Results and Evaluation	48
6.1	Training Trajectory	48
6.2	Aggregate Symbol Error Rate	49
6.3	Error Decomposition	50
6.3.1	Edit-operation breakdown	50
6.3.2	Per-token breakdown	51
6.3.3	SER versus sequence length	51
6.4	Qualitative Examples	52
6.5	Model Weight Distribution	53
6.6	The Synthetic-to-Real Domain Gap	53
6.6.1	Fine-tuning results	54
6.6.2	Qualitative inference on a complete Real Book page	54
6.7	Summary	55
7	Sustainability and Social Commitment	56
7.1	Self-Assessment Summary	56
7.2	Sustainability Matrix	57
7.2.1	Economic dimension	57
7.2.2	Environmental dimension	58
7.2.3	Social dimension	59
7.3	Commitment and Ethical Position	59
7.4	Final Reflection	60
8	Conclusions and Future Work	62
8.1	Conclusions	62
8.2	Future Lines of Research	63
	References	65
A	Additional Tables and Hyperparameters	68
B	Qualitative Predictions	69

C Code Snippets

70

List of Figures

1.1	The C major scale (C4–C5) on the treble staff, rendered in LilyJAZZ notation. Middle C (C4) sits on the first ledger line below the staff; the remaining notes step upward through the lines and spaces, mapping each vertical position to a distinct pitch.	3
1.2	Note duration values rendered in LilyJAZZ notation, from left to right: whole, half, quarter, eighth, 16th, 32nd, dotted quarter, and dotted eighth. The open notehead with no stem encodes the longest value (whole note); each flag added to the stem halves the duration; an augmentation dot extends it by half.	4
1.3	A four-bar jazz lead-sheet excerpt in C major (4/4), rendered in LilyJAZZ style—the same engraving used to generate the synthetic training dataset. Chord symbols above the staff (Cmaj7, Am7, Dm7, G7) encode the I–vi–ii–V harmonic progression; the melody line encodes pitch through vertical position and duration through notehead shape and stem.	5
1.4	Excerpt from <i>The Real Book</i> (“Black Orpheus”, L. Bonfá), showing the hand-engraved notation the OMR system must transcribe: (1) Header metadata. (2) Chord symbol regions targeted by the OCR stream. (3) Monophonic staff systems targeted by the CRNN-CTC model.	6
2.1	Classical OMR pipeline (Bainbridge and Bell, 2001).	11
2.2	Classical OMR pipeline (Rebelo et al., 2012).	11
2.3	CRNN-CTC pipeline for monophonic OMR.	12
2.4	Encoder-decoder transformer architecture for OMR (SMT style).	13
3.1	Gantt chart for the 21-week project (February–June 2026), grouped by work package. Red diamonds mark sprint reviews (M1–M4) and the final defence (M5). Concurrency is limited to naturally compatible task pairs (for instance, evaluation-framework development during the first training run).	23
3.2	Risk-impact matrix locating each of the six register entries on the likelihood/impact plane.	26
4.1	Five-stage inference pipeline.	30
4.2	CRNN-CTC melody recogniser design.	33
5.1	Data pipeline: PrIMuS semantic files are re-rendered in LilyJAZZ, converted to LMX labels, augmented to simulate scanned conditions, and indexed into a vocabulary file.	39
5.2	One clean LilyJAZZ render and five scan-simulated copies produced by the augment stage.	40
5.3	Two-stream inference flow. Music recognition and chord recognition run as independent CRNN-CTC streams; their outputs are joined per staff and validated by the grammar fixer before serialisation as JSON.	44

- 6.1 Training trajectory of the synthetic-data baseline. **Left:** CTC loss on the train (blue) and validation (orange) splits over the 49 epochs that ran before early stopping triggered at patience 12. The train loss continues to decrease while the validation loss plateaus near 0.06[**TODO: verify plateau value against training_log.csv**], indicating a small but bounded overfit margin. **Right:** validation Symbol Error Rate over the same epochs. The curve drops sharply from $\approx 12\%$ to below 2% within ten epochs and converges to $\approx 1.3\%$. The best checkpoint, marked by the vertical dashed line, corresponds to epoch 37; later epochs do not improve validation SER and the early-stopping mechanism eventually halts training. 48
- 6.2 Per-sample SER distribution on the scanned test split, overlaid with its empirical CDF. The aggregate SER of 0.0128 is dominated by a long but thin right tail of high-error samples; the bulk of the distribution sits at zero. Approximately 71 % of test samples are transcribed with zero edits and 95 % fall below SER 0.05. . . . 49
- 6.3 Distribution of edit operations over the scanned test split. Deletions and insertions together account for $\approx 95\%$ of the edit budget; substitutions are a small minority. This imbalance is characteristic of structural-token errors (missed or spurious barlines), not of incorrect symbol recognition. 50
- 6.4 Top-15 token-level edit operations on the scanned test split. **Left:** most frequent substitutions, dominated by duration confusions (16th $\leftrightarrow$ 32nd, eighth $\leftrightarrow$ 16th). **Middle:** most frequent deletions, dominated by missed **measure** tokens (over 1600 occurrences). **Right:** most frequent insertions, again dominated by spurious **measure** tokens. The two structural categories (**measure** and the tie tokens) together account for approximately 87 % of the edit budget. 51
- 6.5 Per-sample SER as a function of ground-truth sequence length on the scanned test split. Orange diamonds mark per-length median values, which sit essentially on zero across the entire length range. The blue cloud above the baseline shows the long-tail samples whose SER lies above the per-length median; there is a mild increase in tail thickness around 120-token sequences but no systematic divergence at long lengths. The CTC alignment remains robust over the full range of sequence lengths in the corpus. 52
- 6.6 A representative perfect prediction on the scanned test split: edit distance 0, SER 0. The input (top) is a typical scan-augmented monophonic staff with beamed eighth and sixteenth notes, a one-flat key signature, and dense rhythmic content; the rendered prediction (bottom) matches it note-for-note. Approximately 71 % of test samples fall in this category. 52
- 6.7 A representative near-median prediction with edit distance 2 and SER 0.038. Errors at this level typically consist of one or two missed or extra structural tokens (barlines or ties) and leave the melodic content intact: in the example above, the pitches and durations are reproduced correctly, but the rendered prediction inserts a spurious barline within the third measure of the input. 53
- 6.8 Marginal weight distributions of the four principal parameter groups of the trained CRNN at epoch 37. All four are close to centred Gaussian with standard deviations in the $[0.17, 0.28]$ range; AdamW’s 1×10^{-4} weight decay keeps the magnitudes bounded. The small spikes near ± 1 in the **fc** panel are an artefact of the classifier saturating on the CTC blank class and the most frequent token. Biases (bottom right) are slightly left-skewed, consistent with the negative-log-likelihood objective preferring to push logits down for rare classes. 54

List of Tables

2.1	Principal labelled corpora used in modern OMR research. PrIMuS is the only one specifically designed for monophonic single-staff training; it is the foundation of the synthetic dataset constructed in this work.	14
3.1	Qualitative comparison of architectural alternatives against the six project criteria. Symbols denote relative suitability: ++ excellent, + good, − poor, -- very poor. . . .	18
3.2	Task summary with effort estimates, dependencies, and principal resources. . . .	22
3.3	Risk register with alternative plans and estimated impact. L/I are likelihood and impact, rated L/M/H.	25
3.4	Roles considered for the project and their loaded hourly rates.	26
3.5	Human-resources budget by work package.	27
3.6	General costs.	27
3.7	Incidentals derived from the risk register. Each entry is the cost of activating the corresponding alternative plan, weighted by the risk’s likelihood.	28
3.8	Total project budget.	28
5.1	Principal libraries used in each layer of the system. Versions are pinned in <code>pyproject.toml</code> ; Poetry produces a reproducible virtual environment across machines.	37
5.2	The <code>albertations</code> transform pipeline applied during the <code>augment</code> stage. Probabilities and parameter ranges are calibrated against a sample of real Real Book scans.	40
5.3	Default hyperparameters of the <code>Config</code> dataclass. Every value is overridable via a CLI flag; the entire <code>Config</code> is serialised alongside the checkpoint so that evaluation always uses the same settings as training.	42
5.4	CLI subcommands exposed by <code>src/cli.py</code> . Every subcommand prints a self-contained help text when invoked with <code>-help</code>	46
6.1	Headline test-set results on the epoch-37 checkpoint (<code>models/latest/best_model.pt</code>), greedy CTC decoding, 4604 held-out samples per split.	49
7.1	Sustainability matrix for the project. PPP refers to the design-and-development stage; the useful-life column refers to deployment and downstream impact.	57

Chapter 1

Introduction and Context

Music notation is one of the most sophisticated symbolic languages ever devised: a two-dimensional graphical system capable of encoding pitch, rhythm, dynamics, phrasing, and expressive intent with sufficient precision to allow performers separated by centuries to reproduce a composer’s intentions from a printed page. Yet the overwhelming majority of notated music exists today only as paper or low-resolution scanned image, inaccessible to the computational tools that now drive music education, music information retrieval (MIR), computational musicology, and digital archiving. *Optical Music Recognition* (OMR) is the discipline that bridges this divide: it translates images of musical scores into structured, machine-readable representations in the same way that Optical Character Recognition (OCR) transformed printed text into searchable, editable data [1].

This thesis describes the design, implementation, and quantitative evaluation of an end-to-end OMR system tailored to a concrete and practically significant target: the monophonic melody lines and chord symbols of jazz lead sheets, with a primary focus on *The Real Book*, the iconic fake-book in the hands of most working jazz musicians worldwide.

1.1 Motivation

1.1.1 The Digitization Gap in Music

Music archives, educational institutions, and individual musicians hold enormous quantities of repertoire that exists only in paper or low-resolution scanned form. Public libraries, conservatory collections, and private fake-book editions contain hundreds of thousands of pages of notation that cannot be searched, transposed, played back, or analysed without manual transcription into notation software. Manual transcription is slow and demands specialist training that is unavailable at scale—per-page effort estimates vary widely by source complexity, but they consistently dominate the cost of corpus-level digitisation projects [2], [3].

The practical consequences are significant. A music educator who wishes to assign a jazz standard in a different key must either own commercial software with the piece already encoded, transcribe it manually, or distribute a photocopy that students cannot edit. A musicologist studying harmonic patterns across a large corpus cannot apply statistical methods to scanned PDFs without first converting them to machine-readable format. A musician practising from a worn photocopy of *The Real Book* cannot feed the page to a practice app that transposes the melody to a B♭ instrument.

OCR technology began solving an equivalent problem for text decades ago, making printed books, newspapers, and documents searchable and editable at almost zero marginal cost. OMR is the musical analogue, but it remains far less mature: the visual language of music notation is substantially more complex than alphabetic text, the available annotated data is orders of magnitude smaller, and the community of researchers and tools is correspondingly narrower—it

is a more specialised field.

1.1.2 Jazz Lead Sheets and *The Real Book*

Notated music exists in many formats—full orchestral scores, piano reductions, chord charts, tablatures—in the same way written language ranges from novels to tweets to legal documents. Among these, jazz lead sheets occupy a particular position. A lead sheet is a compact score format that encodes only the most essential musical information: a single-voice melody line written on a staff, chord symbols printed above it indicating the harmonic context, and—occasionally—lyrics beneath the staff. This format is the lingua franca of jazz practice: it allows musicians to memorise and improvise over a tune from a minimal, portable representation without the burden of a full orchestral score [4].

The Real Book is the most widely circulated collection of jazz lead sheets [5]. Originally assembled as an unauthorised compilation by Berklee College of Music students in the 1970s, it circulated as a hand-copied samizdat for years before receiving official publication. It contains several hundred of the most important jazz standards—tunes by Miles Davis, Thelonious Monk, Bill Evans, John Coltrane, and dozens of other canonical figures—and is found on the music stands of jazz musicians at virtually every level of practice, worldwide.

Despite its cultural centrality, the content of *The Real Book* remains almost entirely locked in paper or scanned-image form. The officially published editions exist as PDFs, but those PDFs are image-based: the musical content is not encoded as symbolic data, and standard music notation software cannot interpret them directly. Digitising *The Real Book*—and the broader class of jazz lead sheets it represents—would unlock a range of practical applications: automatic transposition for Bb, Eb, and F instruments; instant MIDI playback for practice and accompaniment; corpus-level harmonic analysis for musicologists; improved search and cataloguing in digital music libraries; and accessible preparation of educational materials for jazz educators.

1.1.3 Why Existing OMR Tools Fall Short

General-purpose OMR systems have advanced significantly in recent years [1], but they are designed and evaluated primarily on classical full-score notation in standard engraving styles (Finale, Sibelius, Dorico, or LilyPond’s default font). Lead sheets diverge from this baseline in three ways that, together, define the specific challenges this thesis addresses.

Engraving style. Many editions of *The Real Book* use fonts and layouts that deliberately imitate hand-lettered notation. Models trained on classical-style corpora such as PrIMuS [6] or DeepScoresV2 [7] do not transfer naturally to this visual register.

Chord symbols. Jazz chord symbols appear above the staff as free-form text and are a core part of lead-sheet semantics (see [Section 1.2.5](#) for the full grammar). Standard OMR pipelines transcribe pitch and rhythm from the staff; they either ignore chord symbols or treat them as noise. Any lead-sheet OMR system must include a complementary chord recogniser coordinated with melody transcription.

Absence of labelled data. To the best of the author’s knowledge, no publicly available, labelled dataset of Real Book pages with ground-truth symbolic transcriptions exists as of 2026. Supervised training of modern deep models without such a corpus requires synthetic data generation and domain adaptation, whose calibration directly governs the performance of the resulting system. These gaps are surveyed in detail in [Chapter 2](#).

1.2 Musical Notation Fundamentals

This section provides a self-contained introduction to the elements of Western staff notation relevant to this thesis. It is intended for readers with a computer science background who may not

have formal training in music theory. Familiarity with this material is necessary to understand the vocabulary design, the data pipeline, and the error analysis presented in later chapters.

1.2.1 The Staff and the Treble Clef

Western music is written on a *staff*: a set of five equally spaced horizontal lines and the four spaces between them. Pitches are represented by the vertical position of a notehead on this grid: each line and space corresponds to a specific pitch, and moving a notehead one position upward (one line to the adjacent space, or one space to the adjacent line) raises the pitch by one diatonic step.

The absolute pitch assigned to each staff position depends on the *clef* symbol placed at the left edge of the staff. The most common clef in Western notation is the *treble clef* (also called the *G clef*), which curls around the second line from the bottom and designates that line as G4—the G above middle C. With this anchor established, the remaining lines read, from bottom to top, E4, G4, B4, D5, F5; the spaces between them read F4, A4, C5, E5.

When a pitch falls above or below the range of the five-line staff, short horizontal *ledger lines* extend the staff as needed. Middle C (C4), for example, sits on the first ledger line below the treble staff; a high A (A5) sits on the first ledger line above it. Jazz lead sheets use the treble clef exclusively, and this thesis targets the treble clef only (see [Section 1.6](#)).

[Figure 1.1](#) illustrates the ascending C major scale from middle C (C4) to C5 on the treble staff. The leftmost note sits on a ledger line below the staff; successive notes step upward through the five-line grid, making the pitch–position correspondence explicit.

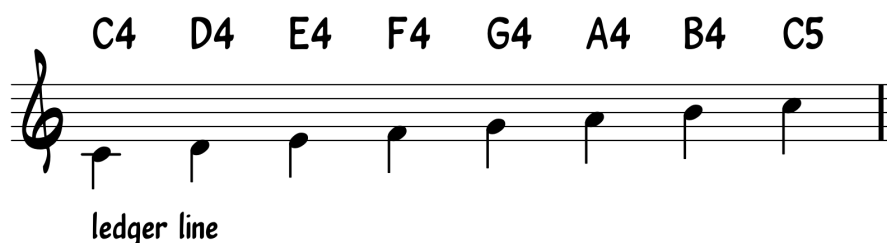


Figure 1.1. The C major scale (C4–C5) on the treble staff, rendered in LilyJAZZ notation. Middle C (C4) sits on the first ledger line below the staff; the remaining notes step upward through the lines and spaces, mapping each vertical position to a distinct pitch.

1.2.2 Pitch, Accidentals, and Key Signatures

Western tonal music divides the octave into twelve chromatic semitones. The seven diatonic pitch classes—A, B, C, D, E, F, G—correspond to the white keys of a piano keyboard and to the named staff positions described above. The five remaining semitones are represented using *accidentals*: a *sharp* ($\sharp$) raises a note by one semitone; a *flat* ($\flat$) lowers it by one semitone; a *natural* ($\natural$) cancels a preceding sharp or flat. Accidentals appear immediately to the left of the notehead they modify and remain in effect for the remainder of the bar in which they appear.

A *key signature*—placed at the beginning of each staff, immediately after the clef—specifies the tonal context of the piece by indicating which pitch classes are to be systematically sharpened or flattened throughout. A key signature of two sharps, for example, signals the key of D major and means that every F and every C should be played as F $\sharp$ and C $\sharp$ unless explicitly cancelled by

a natural sign. Key signatures range from seven flats ($C\flat$ major) to seven sharps ($C\sharp$ major), passing through C major, which carries no accidentals.

1.2.3 Note Duration and Rhythm

The *duration* of a note is encoded by a combination of three visual features:

Notehead shape. A hollow (open) notehead indicates a half note or whole note; a filled (solid) notehead indicates a quarter note or any shorter value. A whole note has no stem; all shorter notes have a vertical stem attached to the notehead.

Flags and beams. Each flag attached to a stem halves the duration of the note relative to a plain stemmed notehead: a single flag denotes an eighth note; two flags a sixteenth note; three flags a thirty-second note. When two or more short notes appear consecutively, their flags may be replaced by a horizontal *beam* connecting their stems, which clarifies the rhythmic grouping within the bar without changing the individual durations.

Augmentation dots. A dot placed immediately to the right of a notehead extends the note's duration by half its original value. A dotted quarter note therefore lasts for one and a half beats in common time; a dotted eighth note lasts for three sixteenth-note durations. Multiple dots are possible (each adding half the preceding augmentation), though double dots rarely appear in jazz lead sheets.

Rests indicate periods of silence and follow the same duration hierarchy as notes, with distinct graphical symbols for each value. A *whole-bar rest*—a filled rectangle hanging below a staff line—indicates that an entire measure is silent, regardless of the time signature.

Figure 1.2 shows the seven standard note values side by side, together with dotted variants. The progressive reduction in note value is visually encoded by the head shape (open vs. filled), stem flags, and augmentation dot.



Figure 1.2. Note duration values rendered in LilyJAZZ notation, from left to right: whole, half, quarter, eighth, 16th, 32nd, dotted quarter, and dotted eighth. The open notehead with no stem encodes the longest value (whole note); each flag added to the stem halves the duration; an augmentation dot extends it by half.

1.2.4 Time Signatures and Measures

The staff is divided by vertical *barlines* into segments called *measures* (or *bars*). Each measure contains a fixed total duration determined by the *time signature*, which appears at the beginning of a piece—and again whenever it changes—as a pair of stacked numbers. The upper number specifies how many beats are in each measure; the lower number specifies which note value counts as one beat (4 for a quarter note, 8 for an eighth note, 2 for a half note).

The time signature 4/4 (“common time”) therefore assigns four quarter-note beats to every measure, while 3/4 assigns three, and 6/8 organises six eighth notes into two compound beats of

three. The jazz repertoire is overwhelmingly written in 4/4, though 3/4 (waltz), 2/2 (cut time), 6/8, 6/4, 2/4, 5/4, and 12/8 all appear in the *Real Book*; these eight time signatures constitute the in-scope set of this thesis (see [Section 1.6](#)).

1.2.5 Chord Symbols

Jazz notation supplements the melodic staff with *chord symbols* printed above the staff at positions corresponding, approximately, to the beat on which each harmonic change takes effect. These symbols specify the harmonic accompaniment and guide the rhythm section and improvising soloists.

A chord symbol encodes: (1) a *root pitch class* (A through G, with an optional flat or sharp); (2) an optional *quality abbreviation* specifying the chord type (e.g. **maj7** for major seventh, **m7** for minor seventh, **7** for dominant seventh, **m7b5** for half-diminished, **dim7** for fully diminished); (3) optional *extensions and alterations* modifying specific chord tones (e.g. **#9**, **b5**, **13**); and (4) an optional *slash bass note* indicating a non-root bass (e.g. **C/E** = C major chord over an E in the bass). Representative examples from *The Real Book* include: **Cmaj7**, **F#m7b5**, **D7#9**, **Bb13**, **Am7/D**.

Chord symbols are typeset as free-form text—not as graphical notation symbols—and their recognition requires optical character recognition (OCR) rather than the staff-pattern recognition applied to the melody line. This distinction motivates the two-stream architecture described in [Chapter 4](#).

1.2.6 The Lead-Sheet Format

A complete jazz lead sheet consists of: (1) one or more systems of single-voice treble-clef staves carrying the melody, with clef, key signature, and time signature at the start of each system; (2) chord symbols above each staff, roughly aligned with the beat at which each harmonic change takes effect; and (3) occasionally, structural markers such as repeat signs, section labels (*A*, *B*, *Bridge*), and tempo or style indications—though these are absent from many informal editions, including large parts of *The Real Book*.

[Figure 1.3](#) illustrates a clean synthetic rendering of a lead-sheet excerpt in the LilyJAZZ style used throughout this project, and [Figure 1.4](#) shows an actual page from *The Real Book* with its key notation elements identified.



Figure 1.3. A four-bar jazz lead-sheet excerpt in C major (4/4), rendered in LilyJAZZ style—the same engraving used to generate the synthetic training dataset. Chord symbols above the staff (**Cmaj7**, **Am7**, **Dm7**, **G7**) encode the I–vi–ii–V harmonic progression; the melody line encodes pitch through vertical position and duration through notehead shape and stem.

1.3 Problem Formulation

The goal of this thesis is to build a system that, given a PDF or image of a jazz lead sheet, automatically produces a structured, machine-readable transcription of its contents: the sequence of notes, rests, and rhythmic markers making up the melody, and the chord symbols specifying the harmonic progression. Despite decades of OMR research, reliable automated transcription of

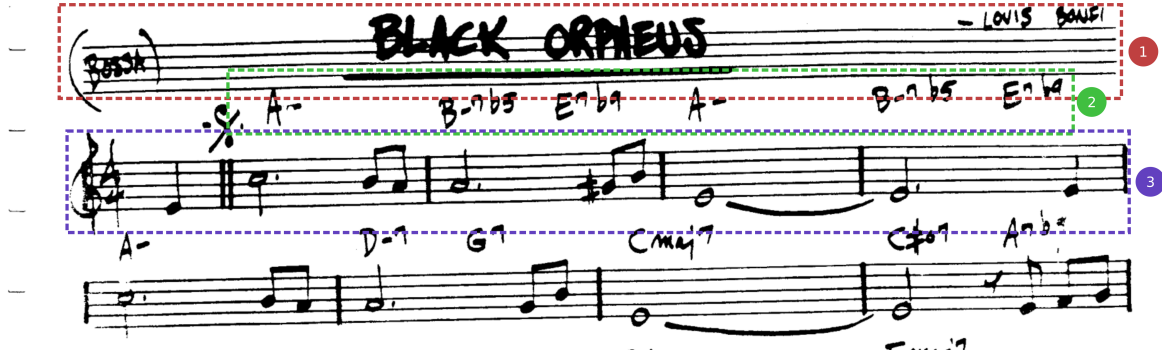


Figure 1.4. Excerpt from *The Real Book* (“Black Orpheus”, L. Bonfá), showing the hand-engraved notation the OMR system must transcribe: (1) Header metadata. (2) Chord symbol regions targeted by the OCR stream. (3) Monophonic staff systems targeted by the CRNN-CTC model.

real-world scores remains an open problem [1]. The challenge has several compounding sources of difficulty.

Notation complexity. Unlike alphabetic text, where symbols are largely independent and arranged in a single linear sequence, musical notation encodes information jointly across two spatial dimensions. A note’s pitch is determined by its vertical position on the staff *relative to the clef*, not by the symbol shape alone; its duration is determined by a combination of head shape, stem presence, flag count, and dot presence. Beams create joint dependencies across groups of notes; ties extend a single pitch across a barline. Any recognition system must capture these two-dimensional, context-dependent symbol interactions without relying on explicit per-symbol segmentation at inference time.

Domain gap. The most widely used annotated OMR datasets—including PrIMuS [6] and DeepScoresV2 [7]—consist of cleanly computer-engraved images rendered at high resolution with standard engraving fonts. Real-world documents, by contrast, are photographed under imperfect lighting conditions, and suffer from scanner noise, perspective distortion, ink bleed, paper yellowing, and JPEG compression artefacts. A model trained exclusively on clean synthetic data generalises poorly to real scans: the feature distribution shifts substantially when crisp black symbols on a white background are replaced by blurred, uneven, noisy images with variable contrast. Bridging this *domain gap* without real annotated training data requires a scan-simulation augmentation pipeline that teaches the model to be invariant to scan-related distortions.

Domain specificity and data scarcity. As argued above, jazz lead sheets use an engraving style for which no publicly labelled corpus currently exists. Training a model for this domain therefore requires constructing a synthetic dataset from scratch using a rendering pipeline tuned to the jazz style, since no off-the-shelf labelled corpus is available.

Dual-stream recognition. The lead-sheet format requires simultaneously processing two distinct visual streams from the same image region: the melodic staff content, which calls for pattern-based sequence recognition, and the chord symbols above the staff, which call for text OCR. These two streams require different feature representations and must ultimately be integrated into a coherent structured output. A complete lead-sheet OMR system must solve both sub-problems and align their results.

This thesis addresses all four difficulties jointly: by constructing a synthetic training corpus in the jazz engraving style, applying a calibrated scan-simulation augmentation pipeline, training a CRNN-CTC model for melody recognition, and building a separate OCR pathway for chord symbols that is fused into a unified output by the inference pipeline.

1.4 Objectives

The **general objective** of this project is to develop, evaluate, and document a proof-of-concept end-to-end OMR system for monophonic jazz lead sheets, and to demonstrate that a CRNN-CTC architecture trained on synthetic data can bridge the domain gap between clean rendered images and real scanned pages.

The **specific objectives** are:

1. Survey the state of the art in OMR, covering traditional modular pipelines, CRNN-CTC sequence models, and transformer-based end-to-end approaches, and justify the architectural choices made in this project.
2. Design and implement a synthetic training dataset by rendering excerpts from the PrIMuS corpus [6] in a jazz engraving style using LilyPond and the LilyJAZZ font, paired with ground-truth annotations in the LMX token format [8].
3. Design and implement a scan-simulation augmentation pipeline calibrated against real Real Book scan characteristics, to close the domain gap at training time without requiring manually labelled real-world data.
4. Design, implement, and train a CRNN-CTC model for monophonic staff-line recognition, and evaluate it rigorously using the *Symbol Error Rate* (SER) [9] metric.
5. Perform quantitative error analysis to characterise systematic failure modes and guide iterative data and model refinements.
6. Build a complete inference pipeline—encompassing PDF input handling, staff detection, melody recognition, grammar correction, and chord symbol extraction—that returns structured JSON output suitable for downstream musical applications.
7. Quantify the domain gap between synthetic and real-scan performance, and reduce it via a fine-tuning stage on a small set of manually annotated Real Book pages.

1.5 Proposed Approach

The core design choice of this thesis is a *CRNN-CTC* architecture: a Convolutional Recurrent Neural Network trained with the Connectionist Temporal Classification (CTC) loss. This model class is well-matched to the left-to-right, variable-length token sequences that arise from reading a single staff line from left to right: the convolutional backbone extracts local visual features along the width of the staff image; two bidirectional LSTM layers model long-range sequential dependencies; and CTC decoding maps the resulting frame-level distributions to a token sequence without requiring explicit segmentation of the image into individual symbols.

An alternative—transformer-based end-to-end models such as the Sheet Music Transformer [10] or the OLIMPIC system for pianoform music [11]—offers higher modelling capacity but demands substantially larger labelled datasets and compute budgets than are available within the scope of a Bachelor’s thesis. Traditional heuristic pipelines [12] avoid data requirements but accumulate segmentation errors that propagate irreversibly through the recognition chain. The CRNN-CTC architecture represents the best trade-off for this project’s constraints; this judgement is substantiated by the comparative analysis in [Chapter 3](#).

The system is organised around a five-stage inference pipeline—preprocessing, staff detection, melody recognition, grammar correction, and chord OCR—whose outputs are merged into a structured JSON document. The decomposition, its contracts between stages, and its design rationale are presented in detail in [Chapter 4](#).

The training strategy also deserves mention. In the absence of labelled real-world data, the model is first trained entirely on synthetic LilyJAZZ-rendered images augmented with scan simulation, then fine-tuned on a small annotated set of real Real Book pages. This two-phase approach is a standard domain adaptation technique and is the practical solution to the data scarcity problem described in [Section 1.3](#).

1.6 Scope

The project is bounded by the following scope definition.

In scope:

- Monophonic (single-voice) melody lines in treble clef, as found in jazz lead sheets.
- Synthetic training data construction via LilyPond rendering of PrIMuS excerpts in the LilyJAZZ engraving style.
- Scan-simulation augmentation modelling noise, blur, perspective distortion, ink bleed, and photometric variation.
- Staff region detection as a preprocessing module.
- Design, training, and evaluation of a CRNN-CTC model for single-staff melody recognition.
- Chord symbol extraction via an OCR-based module.
- Quantitative evaluation using SER and qualitative error analysis.
- Domain-gap analysis and fine-tuning on a small set of annotated Real Book pages.
- A FastAPI web service exposing the full pipeline as a REST endpoint.

Out of scope:

- Polyphonic transcription or simultaneous multi-voice recognition.
- Full-page layout analysis and reading-order recovery across multiple stave systems beyond the per-staff processing implemented.
- Handwritten scores (as opposed to printed or typeset notation).
- Non-Western notation systems or extended contemporary notation.
- Real-time or mobile-device deployment.
- Training large-scale transformer models from scratch.
- Multi-page PDF processing (currently limited to page 0 of any multi-page document).
- Tuplets, repeat barlines, voltas, and navigation marks (D.C., D.S., segno, coda), which do not appear frequently enough in the in-scope repertoire to justify the vocabulary and grammar extensions they would require.

1.7 Document Structure

The remainder of this document is organised as follows.

[Chapter 2](#) surveys the state of the art in OMR, covering the progression from heuristic modular pipelines through CRNN-CTC models to modern transformer-based end-to-end architectures, and reviews the available datasets and symbolic representations relevant to this work.

[Chapter 3](#) covers project management: the justification of the chosen architecture through comparative analysis, the work-breakdown structure and timeline, the budget and economic analysis, and the project’s sustainability and ethical impact assessment.

[Chapter 4](#) describes the conceptual design of the system: the pipeline decomposition, the data strategy, the LMX output representation and vocabulary, the CRNN-CTC architectural choices, the two-stream approach for melody and chord recognition, the training strategy, and the evaluation design.

[Chapter 5](#) describes the implementation: the technology stack, the repository structure, the data and augmentation pipeline, the model and training loop, the chord OCR network, the inference pipeline, the CLI and web service, and the mechanisms used to keep the experiments reproducible.

[Chapter 6](#) presents experimental results: training curves, SER on the synthetic test set, ablation studies, qualitative error analysis by failure category, and domain-gap measurements with and without fine-tuning on real Real Book pages.

[Chapter 7](#) revisits sustainability and ethical considerations in light of the completed work.

Finally, [Chapter 8](#) draws conclusions, reflects on the extent to which the stated objectives were achieved, and outlines directions for future work.

Chapter 2

State of the Art

OMR has evolved through three broadly distinguishable generations: the heuristic modular pipelines that defined the field from the 1990s through the early 2010s; the convolutional–recurrent sequence models trained with Connectionist Temporal Classification (CRNN–CTC) that arose with deep learning; and the most recent end-to-end transformer architectures, which scale to polyphonic and full-page input at the cost of substantially larger labelled corpora. This chapter surveys those paradigms, reviews the principal datasets and symbolic output representations used in the literature, and finally positions this thesis with respect to recent work that targets jazz lead sheets specifically. The two published surveys of the field [1], [2] serve as the structural backbone for the discussion.

2.1 The Classical OMR Pipeline

Until the mid-2010s, almost every OMR system was a modular pipeline of four sequential stages: image preprocessing (binarisation, deskewing); staff-line detection and removal; segmentation and classification of the musical primitives left behind once the staff lines have been erased; and semantic reconstruction, in which the recognised primitives are assembled into a structured score [2]. [Figures 2.1](#) and [2.2](#) show two influential illustrations of this decomposition. Both predate the deep-learning shift discussed in [Section 2.2](#) and share the same backbone of preprocessing, staff handling, primitive recognition, and semantic reconstruction.

The second stage was historically the most studied because every subsequent step depended on it. Dalitz et al. [12] compared the principal families of staff-removal algorithms and contributed a skeletonisation-based method; the *CVC-MUSCIMA* corpus [15] was constructed expressly to benchmark this stage.

Audiveris [16] is the most widely deployed open-source modular OMR engine. Its processing chain combines morphological operations for staff and beam handling, template matching for fixed-shape symbols, a small convolutional classifier for ambiguous glyphs, and external OCR for lyrics, exporting MusicXML 4. The dominant commercial systems—SmartScore, PhotoScore, and SharpEye—follow the same general decomposition but are proprietary [2].

Two structural weaknesses limit the modular approach for the input class targeted in this thesis. First, errors accumulate irreversibly across stages: a single missed staff line corrupts every downstream pitch assignment for that staff, and a mis-segmented chord symbol cannot be recovered by modules that have already committed to a different reading. Second, the visual heuristics that drive each module are tuned for classical engraving styles; the rounded noteheads, loosely-spaced beams, and hand-lettered chord symbols characteristic of *The Real Book* fall outside the operating envelope of the morphological filters those pipelines rely on. This motivates the move to learning-based, end-to-end recognition.

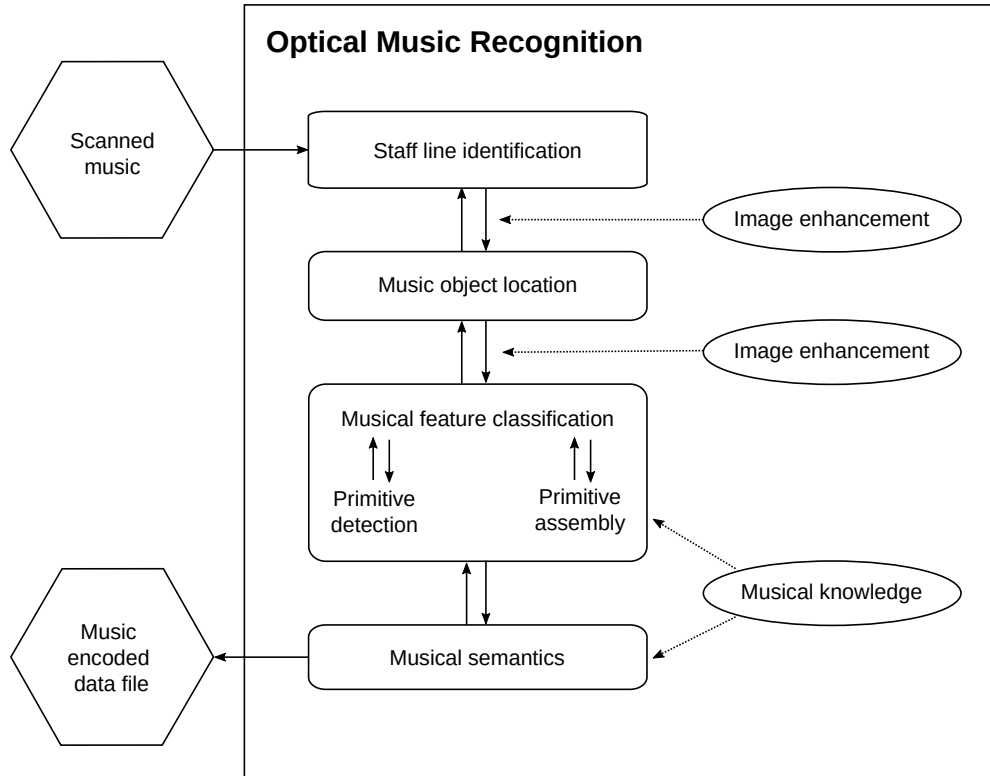


Figure 2.1. Classical OMR pipeline architecture proposed by Bainbridge and Bell [13]. The four-stage structure (preprocessing, staff handling, primitive recognition, and semantic reconstruction) set the template for the field for two decades. Source: Wikimedia Commons, redrawn from the cited work.

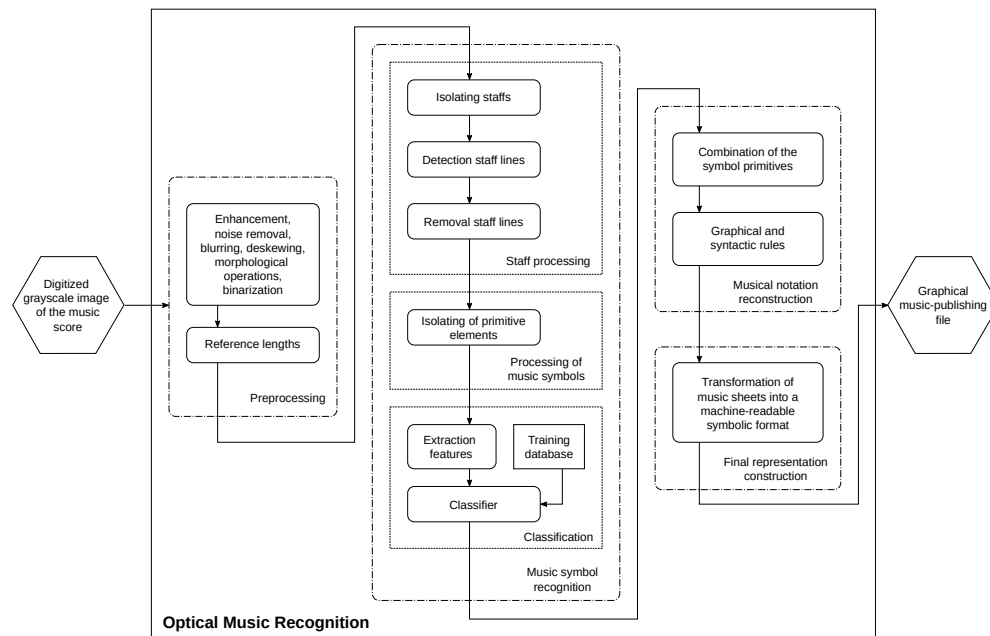


Figure 2.2. The widely-cited OMR block diagram of Rebelo et al. [14], which explicitly shows the feedback loops from musical knowledge into the recognition stages. Source: Wikimedia Commons, redrawn from the cited work.

2.2 Deep Learning Paradigm: CRNN with CTC

The deep-learning era of OMR begins with Calvo-Zaragoza and Rizo [17], who in 2018 applied to single-staff music the CRNN–CTC architecture that had recently become standard for scene-text and handwritten-text recognition. The model treats a staff image as a sequence of vertical strips, extracts local visual features through a convolutional backbone, contextualises them with a bidirectional recurrent network, and decodes the resulting per-frame distribution into a token sequence using the CTC alignment—eliminating the need for per-symbol segmentation.

Alongside the architecture, Calvo-Zaragoza and Rizo released the *PrIMuS* corpus [6], [17]: 87 678 real-music monophonic incipits sourced from the RISM library and distributed in five representations, including two purpose-built token formats. The *agnostic* encoding lists visual primitives without musical interpretation; the *semantic* encoding lists musical events such as `note-C4_eighth` or `keySignature-DM` directly. The CRNN–CTC architecture trained on this corpus achieves Symbol Error Rates near 1 % on the agnostic encoding and the low single digits on the semantic encoding, establishing the de facto baseline for monophonic OMR. A follow-up release—Camera-PrIMuS [18]—augments the same incipits with synthetic photographic distortion and demonstrates that the same network retains useful accuracy on phone-captured real-world inputs. This is the precise template that the present thesis follows. Figure 2.3 illustrates the CRNN–CTC pipeline as applied to a music staff.

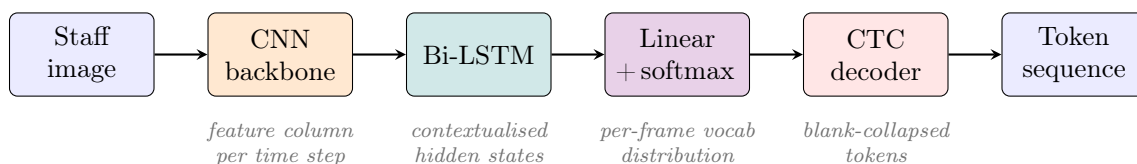


Figure 2.3. The CRNN–CTC pipeline applied to a music staff. The image is treated as a sequence of vertical feature columns extracted by a convolutional backbone (ResNet18 in this work); a bidirectional LSTM contextualises each column; a linear projection produces a per-frame probability distribution over the vocabulary; and CTC decoding collapses the blank-padded alignment into a flat token sequence—without requiring any explicit symbol segmentation. Architecture introduced by Calvo-Zaragoza and Rizo [17].

2.3 End-to-End Transformer Architectures

From 2024 onward the OMR community has converged on attentional encoder–decoder transformers for tasks that exceed the scope of monophonic single-staff recognition. The *Sheet Music Transformer* (SMT) [10] was the first fully transformer-based image-to-sequence OMR system to address polyphonic transcription end-to-end: a vision-transformer encoder produces a flattened patch sequence which a stacked decoder autoregressively maps to the linearised target representation, dispensing with CTC entirely. *SMT++* [19] extends the approach to full-page pianoform input and currently defines the state of the art on that task. *LEGATO* [20]—released in mid-2025—takes the trend further with a vision-LLM backbone, an ABC-notation decoder, and over 214 000 typeset pages of pretraining, reporting absolute error reductions of 47 %–68 % over previous state of the art on hard typeset benchmarks.

Figure 2.4 shows the generic encoder–decoder structure shared by these systems.

These architectures, however, demand resources that exceed the budget of a Bachelor’s thesis. For a strictly monophonic target domain—where the CRNN–CTC baseline already attains sub-percent SER and the headroom for further gain is in domain robustness rather than model capacity—CRNN–CTC remains the right point on the cost/capability curve. The quantitative justification of this choice is developed in Chapter 3.

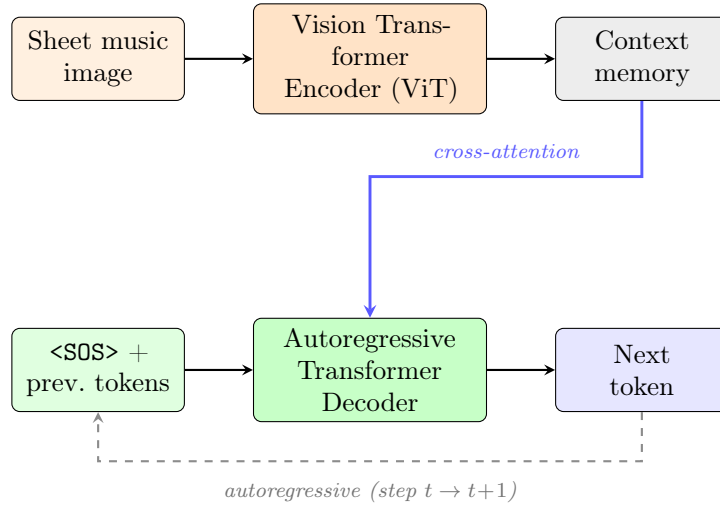


Figure 2.4. The encoder–decoder transformer used in OMR systems such as SMT [10] and SMT++ [19]. A Vision Transformer (ViT) encoder maps the sheet music image to a sequence of patch embeddings; an autoregressive decoder then generates the output token sequence one token at a time, attending at each step to the full encoder memory via cross-attention. Unlike CRNN–CTC, no explicit segmentation or conditional-independence assumption is imposed—the decoder can, in principle, model arbitrary long-range token dependencies.

2.4 Datasets for OMR Training

The choice of architecture is inseparable from the choice of training corpus. Table 2.1 summarises the principal labelled corpora available to the field as of 2026.

Two observations shape the data strategy of this thesis. First, none of these corpora captures the LilyJAZZ-style engraving of *The Real Book*, and none labels chord symbols as first-class objects; the only directly comparable corpus emerged contemporaneously with this work [5] and remains too small to train a CRNN–CTC model from scratch. Second, every recent end-to-end system—SMT, SMT++, OLiMPiC, LEGATO, and Camera-PrIMuS itself—follows the same recipe of synthetic pretraining followed by fine-tuning on a small annotated target set. This is the recipe applied here.

2.5 Symbolic Output Representations

The output of an OMR system is a symbolic description of the score; the choice of representation shapes both the modelling architecture and the achievable error rate. **MusicXML** is the de facto interchange standard between notation editors and preserves polyphony, layout, lyrics, and chord symbols losslessly, but its verbosity inflates sequence length and vocabulary size when fed to a sequence model. **MEI** is even more expressive but inherits the same penalty. **Humdrum **kern** and **ABC notation** are far more compact and are used as targets of recent transformer-based work [10], [20], at the cost of reduced expressiveness for complex multi-voice layout. The PrIMuS **agnostic** and **semantic** formats are compact and CTC-friendly but interoperate only with the PrIMuS ecosystem.

LMX (Linearised MusicXML), introduced by Mayer et al. [11] and maintained as an open specification [8], depth-first-traverses the MusicXML tree and emits one whitespace-separated token per leaf event. It is round-trippable to MusicXML with minimal information loss, substantially more compact than the parent format, and was designed specifically as a CRNN- and transformer-friendly target; the OLiMPiC reference implementation provides a Python **Linearizer/Delinearizer** pair that converts between the two representations directly.

This work adopts LMX with a project-specific grammar that drops features outside the jazz

Dataset	Type	Size	Labels	Use
PrIMuS [6], [17]	Monophonic, typeset	87 678 incipits	agnostic / semantic / MEI	Monophonic CRNN-CTC training
Camera-PrIMuS [18]	Monophonic, distorted	87 678 incipits	same as PrIMuS	Domain-robustness training
DeepScoresV2 [7]	Polyphonic, typeset	255 385 pages, ~151 M symbols	per-symbol bboxes + masks	Music object detection
CVC-MUSCIMA [15]	Polyphonic, handwritten	1 000 pages	staff-removal masks	Staff removal / writer ID
MUSCIMA++ [21]	Polyphonic, handwritten	140 pages, ~91 k symbols	MuNG notation graph	Symbol + relation recognition
DoReMi [22]	Polyphonic, typeset (Dorico)	~6 432 pages, ~1 M objects [TODO: verify: exact page and object counts from DoReMi arXiv 2107.07786]	MusicXML + MuNG-style	Universal OMR research
OLiMPiC [11]	Pianoform, typeset + scan	~17 945 synth, 2 931 scanned [TODO: verify: confirm split counts against Mayer et al. 2024 Table 1]	MusicXML / LMX	End-to-end pianoform OMR

Table 2.1. Principal labelled corpora used in modern OMR research. PrIMuS is the only one specifically designed for monophonic single-staff training; it is the foundation of the synthetic dataset constructed in this work.

lead-sheet domain (tuplets, slurs, articulations, double accidentals, non-treble clefs); the full grammar is described in [Section 4.3](#). LMX was preferred over the PrIMuS native formats because it round-trips losslessly to MusicXML—the format expected by downstream music applications—and because using the same representation as the OLiMPiC reference work [11] keeps the present results directly comparable.

2.6 Evaluation Metrics

Symbol Error Rate (SER) is the de facto evaluation metric for sequence-based OMR. It is defined as the Levenshtein edit distance between the predicted and the reference token sequences, normalised by the reference length [9], mirroring the Word Error Rate of automatic speech recognition. Its principal weaknesses, flagged by both Shatri and Fazekas [1] and by Hajič et al. [3], are that all token-level edits count equally regardless of musical significance and that SER values depend on the chosen tokenisation, so they are not directly comparable across encodings.

Two recent metrics partially address these shortcomings. **TEDn** (normalised tree edit distance on the parsed MusicXML tree) weights structural errors appropriately and is the primary metric in OLiMPiC and SMT++. **OMR-NED** [23] extends SER with per-element categories so that error distribution—not only error rate—is legible from the metric itself. This thesis reports SER as the primary metric for backward comparability and complements it with a category-level error breakdown in [Chapter 6](#).

2.7 OMR for Jazz Lead Sheets

Despite the cultural importance of jazz fake-books, OMR of jazz lead sheets received almost no dedicated attention in the published literature before 2025. Both major surveys [1], [2] list chord-symbol handling among the gaps in the field, and no general-purpose OMR engine treats chord symbols as a first-class output class.

The earliest dedicated work is an undergraduate honours thesis at UCF [4], which built a non-deep modular pipeline for chord-symbol recognition on printed and handwritten lead sheets and used the recovered chord progressions to attempt formal-structure reconstruction. The closest published precedent is **Martínez-Sevilla et al.** [5], presented at ISMIR 2025 and roughly contemporaneous with the final stages of this thesis. That work introduces a 293-sheet dataset of *handwritten* jazz lead sheets with ****kern** and MusicXML ground truth, and trains a transformer-decoder model with symbolic-aware tokenisation, pretrained on synthetic data and fine-tuned on the handwritten target set.

The present work differs in input domain (printed and scanned Real Book pages versus fully handwritten sheets), in architecture (CRNN–CTC versus transformer decoder), in chord handling (a two-stream design with independent staff and chord networks merged in post-processing versus a unified single-stream model), and in output format (LMX versus ****kern**). The two systems are thus complementary points in the design space, and both contribute training corpora, models, and evaluation protocols that future work can compare and combine.

2.8 Summary

Modular pipelines remain useful for full-page typeset classical scores but do not generalise to the visual and semantic peculiarities of jazz lead sheets. End-to-end deep models—CRNN–CTC for monophonic input, transformers for polyphonic and full-page input—now define the methodological frontier and converge on a common recipe of synthetic pretraining followed by fine-tuning on a small annotated target set. Symbolic output formats have moved toward MusicXML-linear encodings (prominently LMX), and evaluation practice is gradually shifting

from raw SER toward structurally aware metrics. Jazz lead-sheet OMR remained essentially open until 2025; this thesis and the concurrent ISMIR 2025 work [5] represent the first two focused attempts to fill that niche. The architectural choices made in this thesis—detailed in [Chapter 4](#)—are motivated by the findings surveyed above.

Chapter 3

Project Management (GEP)

This chapter records the management dimension of the project: the comparative justification of the chosen architecture against the plausible alternatives, the iterative methodology adopted to execute it, the work-breakdown structure and Gantt schedule, the risk register and contingency plans, and the economic budget with its control mechanism. The introductory framing of the project—academic context, scope, objectives, and stakeholders—is given in [Chapter 1](#) and is not repeated here.

3.1 Justification of the Chosen Architecture

3.1.1 Existing alternatives

[Section 2.1](#), [Section 2.2](#), and [Section 2.3](#) surveyed the three principal families of OMR systems in the literature. Considered as candidate architectures for this thesis, they are:

1. **Traditional modular pipelines** (staff detection/removal, symbol segmentation, symbol classification, reconstruction) with the well-known error propagation issues catalogued by Dalitz et al. [\[12\]](#) and the modern survey [\[2\]](#).
2. **CRNN–CTC sequence models**, well-suited to left-to-right staff-line transcription and the de facto baseline for monophonic OMR [\[6\]](#), [\[17\]](#).
3. **Transformer-based end-to-end models**, stronger for full-page and polyphonic settings [\[10\]](#), [\[19\]](#), [\[20\]](#) but with substantially heavier data and compute requirements.

3.1.2 Selection criteria

The decision is based on six criteria reflecting both the technical target and the practical constraints of a Bachelor’s thesis.

Expected accuracy on monophonic lines. The principal deliverable of the project is a measurable Symbol Error Rate on monophonic jazz lead sheets, so any architecture whose published accuracy on monophonic input is substantially below the state of the art is disqualified at the outset.

Computational feasibility on consumer hardware. Training must complete within a few days on a single NVIDIA RTX 3060 with 12 GB of VRAM, the hardware available to the author. Any model whose memory or compute footprint exceeds that envelope either has to be downscaled—defeating its accuracy advantage—or trained in cloud rentals, which is incompatible with the project budget.

Data requirements. The only labelled corpus directly usable for monophonic OMR is PrIMuS [\[6\]](#), with approximately 88 000 incipits; the closest jazz-specific corpus emerged only in mid-2025 and is substantially smaller [\[5\]](#). Architectures whose published results rely on hundreds

of thousands of polyphonic samples are infeasible without a data-acquisition phase that exceeds the scope of this thesis.

Implementation complexity. A single-person project does not have the engineering bandwidth to debug a custom transformer training stack in parallel with a synthetic data pipeline. A well-documented architecture with mature open-source reference implementations is preferred.

Reproducibility within the project’s timeline. Each sprint cycle must terminate with a measurable, reproducible result. Architectures whose training is highly stochastic, or whose hyperparameter space is large enough to require dozens of trial runs before convergence, would consume the project’s iteration budget before producing a stable baseline.

Extension potential for future work. The thesis records the design space available to follow-up work; an architecture that has natural extensions to polyphony, full-page input, or chord-symbol integration is preferred over one that locks the line of research into a single configuration.

Table 3.1. Qualitative comparison of architectural alternatives against the six project criteria. Symbols denote relative suitability: ++ excellent, + good, – poor, -- very poor.

Criterion	Traditional pipeline	CRNN–CTC	Transformer
Monophonic line accuracy	–	+	++
Computational feasibility	++	+	--
Data requirements	++	+	--
Implementation complexity	+	+	--
Reproducibility in TFG timeline	+	++	–
Extension potential (future work)	–	+	++

Table 3.1 shows that CRNN–CTC is the best fit for this stage: it provides enough modelling capacity for monophonic lead sheets while remaining trainable and debuggable within the available timeframe and hardware budget. A transformer would in principle yield higher accuracy on the monophonic line, but the data and compute costs of a from-scratch training run are incompatible with the resources of a single-person project. A traditional modular pipeline is cheap to implement but is penalised by error propagation across stages and by its sensitivity to the jazz engraving style; this option is in fact retained as the *lower bound* baseline against which the CRNN–CTC results are compared in Chapter 6.

3.1.3 Chosen alternative

The selected architecture is a **two-stream CRNN–CTC system** in which a music CRNN–CTC transcribes the monophonic staff line and a second, independently trained CRNN–CTC performs OCR on the chord strip above the staff; its design is the subject of Chapter 4 and its implementation of Chapter 5. The decision balances modelling capacity against feasibility: the model is lighter than transformer alternatives, has strong prior art in OMR datasets, and supports robust sequence prediction without requiring per-symbol segmentation of the input image. Splitting the chord recogniser into a separate network keeps both vocabularies small and lets each branch be trained and debugged independently.

Three concrete consequences follow from this choice. First, the training corpus can be built entirely from a single public dataset (PrIMuS) re-rendered in the target engraving style, which keeps the data-acquisition cost within the budget allocated for the thesis. Second, the output representation can be a compact flat token sequence (LMX, Section 4.3) consumed by both

the music and the chord branches, avoiding the verbose-XML output that transformer-decoder architectures typically produce. Third, the model’s training time on a consumer GPU is short enough (of the order of a day per full run) that several training iterations per sprint are realistic; this is what makes the iterative methodology of [Section 3.2](#) workable in practice.

The decision is also a risk-control choice. A traditional modular pipeline (the lower-bound baseline) can be implemented early and serves as a reference point for measuring the CRNN–CTC’s contribution; the CRNN–CTC model is then iteratively improved through data and training refinements. This staged execution path still yields publishable results even if the more ambitious extensions—fine-tuning on real Real Book pages, chord OCR, full-page inference—cannot all be completed within the available time.

3.2 Methodology

The project follows an **iterative and incremental** methodology with sprint-based checkpoints and supervisor reviews. Each cycle is required to produce measurable artefacts—datasets, training logs, SER values, error analyses—rather than internal milestones whose progress cannot be inspected from the outside. This produces an audit trail that grounds every replanning decision in evidence rather than in subjective progress estimates, and gives the supervisor concrete material to react to at each biweekly meeting.

Execution is explicitly organised as a closed loop between three activities: *data construction* (rendering, augmentation, vocabulary), *model training* (architecture changes, hyperparameter sweeps, training-curve monitoring), and *evaluation and error analysis* (SER measurement, per-token breakdown, qualitative case studies). Each of the three is revisited multiple times during the project: an error analysis that identifies a systematic failure mode triggers a data-pipeline change, which triggers a new training run, whose results feed back into the next error analysis. A naïve waterfall reading of the work-package structure (WP1 → WP2 → WP3 → WP4 → WP5) would mistake the *first* traversal of this loop for the *only* traversal; in practice, sprint reviews regularly send the project back to a prior WP for incremental refinement, and the budget reserves time for that.

Four practices enforce methodological rigour and are applied uniformly across the project.

Traceability. Every milestone artefact—a dataset version, a model checkpoint, an SER measurement—is linked to a specific repository commit and to a documented design decision. This makes it possible to reconstruct, several months after the fact, exactly which version of the data and which version of the code produced any given experimental number. Without traceability, ablation studies and regression checks become unreliable, because there is no fixed reference point against which to compare.

Reproducibility. Data generation, training, and evaluation are all driven by scripts rather than by interactive notebook sessions, and the full `Config` dataclass is embedded in every checkpoint ([Section 5.9](#)). A consequence is that any experiment reported in [Chapter 6](#) can be re-run with a single CLI invocation from a clean clone of the repository on a different machine; this is enforced as Hard Constraint 3 in the project’s contributor guide.

Quantitative control. Symbol Error Rate is measured at every significant training iteration, not only at the end of a sprint. Side metrics—training/validation loss gap, per-token edit-distance breakdown, per-sample SER distribution—are computed alongside, so that a single aggregate number is never the basis for a design decision. The category-level error breakdown of [Section 6.3](#) is the direct product of this practice.

Corrective loop. Each sprint ends with a review that can adjust scope, effort estimates, or risk responses for the next sprint. The review is held with the supervisor and consumes a fixed budget (three hours per sprint, accounted for in [Table 3.2](#)); it produces a short written note on whether the sprint hit its objectives, what evidence supports that judgement, and what is carried forward into the next sprint. The note is the input to the budget-control mechanism described in

Section 3.5.5.

Monitoring tools include Git and GitHub for version control, per-experiment training logs and CSV summaries written into the model directory, and biweekly milestone meetings with the supervisor at which the artefacts produced in the previous sprint are reviewed and the priorities for the next one are agreed.

3.3 Project Planning

3.3.1 Work-breakdown structure

The plan spans 21 weeks (February–June 2026), organised into five sprint-aligned work packages (WP1–WP5) plus a transversal management work package (WP0) that runs in parallel throughout the project. Execution control uses **fixed four-week sprints** aligned with the work packages (each WP1–WP4 is one sprint; WP5 closes the project over five weeks), with a biweekly supervisor check-in inside each sprint. Backlog priorities, risks, and effort estimates are reviewed at every sprint boundary and may be revised in either direction.

The six work packages have the following internal logic.

WP0 – Project management (130h, concurrent). This package contains the activities that span the project rather than belonging to any one technical phase: the writing of the GEP deliverables, the biweekly supervisor meetings, and the continuous writing of the thesis document. It is intentionally allocated a substantial budget (26 % of total effort) because the GEP coursework requires four deliverables in its own right and because deferring thesis writing until the final weeks is one of the principal failure modes of single-person projects.

WP1 – Literature review and baseline (78h). This package establishes the analytical foundation: a literature survey of OMR (the material that becomes [Chapter 2](#)), an exploration of the public OMR datasets that informs the data-strategy choice, and a classical computer-vision baseline that provides a quantitative reference against which the CRNN–CTC results can be compared. The classical baseline also serves a risk-control function: if the CRNN–CTC model fails to converge, the baseline ensures the project still has measurable results to report.

WP2 – Dataset construction (73h). This package builds the synthetic training corpus that the CRNN–CTC requires. The output of WP2 is the LMX vocabulary, the LilyJAZZ rendering pipeline, the scan-simulation augmentation pipeline, and the train/validation/test splits. WP2 must complete before WP3 can begin in earnest, because the model architecture is dimensioned to the vocabulary the dataset produces.

WP3 – CRNN–CTC development (75h). This package designs and implements the model, the training loop, and the evaluation framework. The training-pipeline task (T3.2) overlaps deliberately with the evaluation-framework task (T3.4) because the latter can be developed against synthetic CRNN outputs before the former has fully converged. The package ends with an initial end-to-end training run whose outputs feed directly into WP4.

WP4 – Iterative refinement (68h). This package closes the train–evaluate–improve loop: error analysis identifies systematic failure modes, the data pipeline and the training configuration are adjusted, and the model is retrained. This is the work package where most of the closed-loop behaviour described in [Section 3.2](#) actually happens; its 68-hour budget is split across roughly two sprints of refinement and ablations.

WP5 – Final evaluation and defence (85h). This package contains the final evaluation on held-out data, the qualitative error analysis, a conditional extension task (T5.3, budgeted at 20h and contingent on the monophonic objective having been achieved), the final integration pass on the thesis document, and the defence preparation. The conditional extension task is the most important schedule buffer in the plan: it can be reduced to zero without compromising the core deliverable.

3.3.2 Tasks, dependencies, and resources

Table 3.2 lists the 31 tasks of the plan with their effort estimates, dependencies, and principal resources. The estimates are expressed in hours per task and were validated against the author’s available weekly workload of four to five effective working hours per day, which corresponds to approximately 24 h per week of sustained project effort over the 21-week schedule.

The dependency column distinguishes between strict dependencies—a task cannot begin until its predecessor is complete—and partial dependencies, marked “*partial*”, where a task may begin once its predecessor has reached a sufficient maturity to provide the required input. Partial dependencies are used sparingly, since they are difficult to coordinate in a single-person project where context switching has a real cost.

Concurrency in the schedule is intentionally limited. A single-person project does not benefit from the parallelism that drives multi-person planning, because the same individual cannot productively split attention across more than one major task in the same sprint. Overlap is therefore introduced only when two tasks are naturally compatible: for example, developing the evaluation framework (T3.4) while the first training run (T3.3) is in progress, since training runs in the background on the GPU and the developer is otherwise idle.

3.3.3 Gantt chart

Figure 3.1 shows the full 21-week timeline. The chart is grouped by work package for readability; planning control is performed at the sprint level with a fixed four-week cadence and an inspection/adaptation checkpoint at every boundary. Sprint reviews are marked as red diamonds (M1 through M4), together with the final defence at week 21 (M5).

Three structural features of the chart deserve specific attention. First, the management package (WP0) spans the entire timeline, reflecting the policy of continuous thesis writing and biweekly supervisor meetings; the four GEP-specific tasks (T0.1–T0.4) are concentrated in the first half of the project to align with the GEP coursework deadlines. Second, the technical packages WP1–WP4 are scheduled back-to-back with no slack between them, because each one depends on the deliverable of its predecessor in a way that does not admit overlap. The schedule resilience instead comes from the conditional extension task in WP5, which can be reduced if any of the earlier packages overruns its budget. Third, the bars within each sprint deliberately do not all extend to the sprint boundary; this leaves room within each sprint for the corrective-loop activities of Section 3.2, which would otherwise have to come out of the next sprint’s allocation.

Table 3.2. Task summary with effort estimates, dependencies, and principal resources.

Code	Task	h	Dependencies	Resources
WP0 – Project management (130 h, concurrent)				
T0.1	Context and scope definition	20	–	Zotero
T0.2	Temporal planning	12	T0.1	–
T0.3	Budget and sustainability	15	T0.2	–
T0.4	Final GEP integration	8	T0.3	–
T0.5	Supervisor meetings	25	Concurrent	–
T0.6	Thesis writing	50	Concurrent	L ^A T _E X
WP1 – Sprint 1: literature review and baseline (78 h)				
T1.1	OMR literature survey	25	–	Library access
T1.2	Dataset exploration	15	T1.1 (partial)	–
T1.3	Classical CV baseline	25	T1.2	OpenCV
T1.4	Baseline evaluation	10	T1.3	–
T1.5	Sprint 1 review	3	T1.4	–
WP2 – Sprint 2: dataset construction (73 h)				
T2.1	Output encoding (LMX) design	12	T1.5	music21
T2.2	Synthetic rendering pipeline	35	T2.1	LilyPond
T2.3	Data augmentation	15	T2.2	OpenCV / al- bumentations
T2.4	Validation and splits	8	T2.3	–
T2.5	Sprint 2 review	3	T2.4	–
WP3 – Sprint 3: CRNN–CTC development (75 h)				
T3.1	CRNN architecture design	25	T2.5	Consumer GPU
T3.2	Training pipeline	20	T3.1	–
T3.3	Initial training run	15	T3.2	GPU
T3.4	Evaluation framework	12	T3.2 (partial)	–
T3.5	Sprint 3 review	3	T3.3, T3.4	–
WP4 – Sprint 4: iterative refinement (68 h)				
T4.1	Error analysis	10	T3.5	–
T4.2	Data pipeline improvements	15	T4.1	–
T4.3	Model refinement runs	25	T4.2	GPU
T4.4	Ablation studies	15	T4.3	GPU
T4.5	Sprint 4 review	3	T4.4	–
WP5 – Sprint 5: final evaluation and defence (85 h)				
T5.1	Final evaluation	15	T4.5	GPU
T5.2	Qualitative error analysis	10	T5.1	–
T5.3	Extension exploration (conditional)	20	T5.1	GPU
T5.4	Final thesis writing	30	T5.2	L ^A T _E X
T5.5	Defence preparation	10	T5.4	–
TOTAL		509		

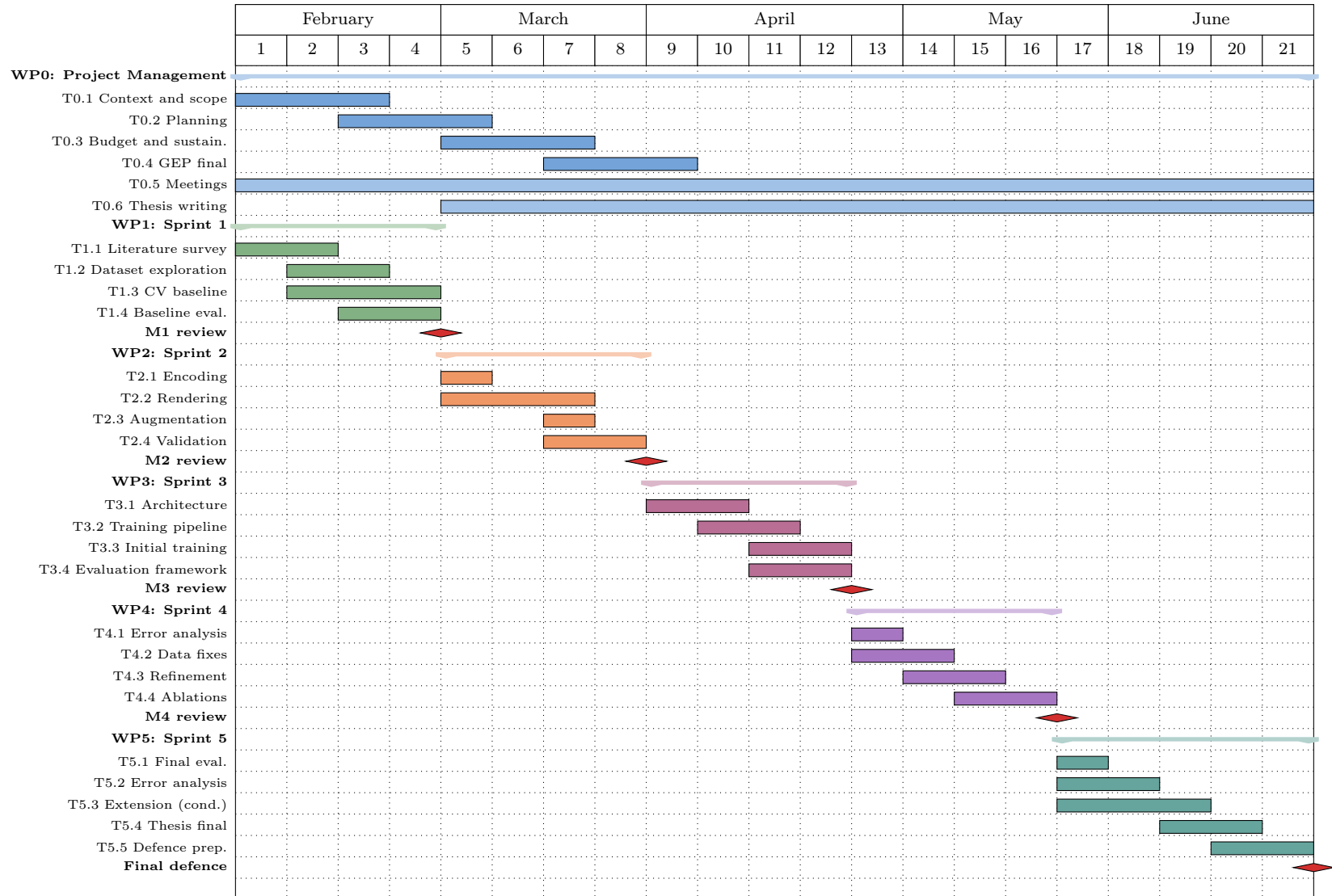


Figure 3.1. Gantt chart for the 21-week project (February–June 2026), grouped by work package. Red diamonds mark sprint reviews (M1–M4) and the final defence (M5). Concurrency is limited to naturally compatible task pairs (for instance, evaluation-framework development during the first training run).

3.4 Risk Register and Contingency Plans

The risk register identifies six principal risks that were foreseeable at the start of the project and for which explicit mitigations or alternative plans can be prepared in advance. Each risk is rated on two axes—likelihood and impact—using a three-level scale (low, medium, high), and is paired with an alternative plan, an estimate of how much schedule the activation of that plan would consume, and an estimate of any additional resources it would require. The register is summarised in Table 3.3; the discussion below explains the reasoning behind each entry.

Limited ground-truth data on real Real Book pages (likelihood high, impact high). The fine-tuning stage of the training plan requires manually labelled scans of real Real Book pages, which the author must produce. If labelling proves slower than estimated—an outcome made more likely by the absence of a similar corpus to bootstrap from—the project falls back to the synthetic LilyJAZZ-augmented corpus alone, accepting that the synthetic-to-real domain gap will remain partly open. The fallback consumes up to ten hours of reallocated effort but requires no additional resources, because the synthetic pipeline is already implemented.

Staff-detection errors propagating downstream (likelihood medium, impact medium). The morphological staff detector can fail on degraded scans, and its failures propagate irreversibly into the CRNN’s input. The fallback is to use manually cropped staff regions for critical evaluations, sidestepping the detector entirely for the purposes of measuring CRNN-only accuracy. This shifts hours from the conditional extension task (T5.3) to the final thesis-writing task (T5.4) but does not require new resources.

Encoding-design complexity (likelihood medium, impact medium). The LMX vocabulary is the only output representation the model can produce; if the initial vocabulary design proves too rich (too many rare classes that the model under-fits) or too restrictive (missing tokens that the dataset emits), training results will be unreliable until the vocabulary is revised. The fallback is to start from a minimal vocabulary and extend it incrementally after the model has reached a stable convergence profile. This consumes up to six hours of redesign within the project buffer and does not propagate further.

Model underfitting or CTC misalignment (likelihood medium, impact high). CTC training is known to fail catastrophically if the input time budget is shorter than the target token sequence, or if the model’s capacity is insufficient for the vocabulary. The fallback is to simplify the vocabulary, retrain on cleaned data subsets, and adjust the image-scale parameter so that the time axis available to the CTC alignment is sufficient. This consumes up to ten hours from the project buffer and requires additional GPU time, accounted for in the incidentals table.

Scope creep toward polyphony (likelihood medium, impact high). A persistent risk in OMR research is the temptation to extend the model to polyphonic input before the monophonic objective has been achieved. The mitigation is structural rather than reactive: the project scope (Section 1.6) explicitly excludes polyphony, and the conditional extension task (T5.3) is restricted to a feasibility discussion rather than implementation. If the temptation arises mid-project, up to eighteen hours of T5.3’s budget can be recovered for the core monophonic deliverable.

Computational resource constraints (likelihood low-to-medium, impact medium). The available hardware (an NVIDIA RTX 3060 with 12 GB of VRAM) fits the baseline model comfortably but provides limited headroom for hyperparameter sweeps. If concurrent obligations occupy the workstation or if a significant resize of the model becomes necessary, the fallback is a short cloud-GPU rental (a single AWS or Lambda Labs session, of the order of €20–50) that lets the project clear the bottleneck without impact on the core schedule. Setup consumes up to five hours and is the only entry in the register that requires direct procurement.

To improve schedule robustness, the plan includes explicit slack in three places: conservative estimation in the two highest-uncertainty tasks (dataset rendering and model refinement); the conditional extension task T5.3, which can be reduced without compromising the core objective; and progressive thesis writing throughout execution rather than the more common pattern of

postponing all writing to the final weeks. Together, these three buffers absorb the schedule consequences of the lower-impact risks without triggering the budget-control mechanism described in [Section 3.5.5](#).

Table 3.3. Risk register with alternative plans and estimated impact. L/I are likelihood and impact, rated L/M/H.

ID	Risk	L/I	Alternative plan	Schedule impact	Extra resources
R1	Limited ground-truth data on real Real Book pages	H/H	Fall back to synthetic / typeset sources and strengthen the augmentation pipeline.	Reallocate up to 10 h.	None.
R2	Staff-detection errors propagating downstream	M/M	Use manually cropped staff regions for critical evaluations.	Shift hours from T5.3 to T5.4.	None.
R3	Output-encoding design complexity (LMX vocabulary)	M/M	Start from a minimal vocabulary and extend incrementally after stable runs.	Up to 6 h redesign.	None.
R4	Model underfitting or CTC misalignment	M/H	Simplify vocabulary, retrain with cleaned subsets, adjust image scale.	Up to 10 h from buffer.	Additional GPU time.
R5	Scope creep toward polyphony	M/H	Restrict extension to feasibility discussion only.	Recover up to 18 h.	None.
R6	Computational resource constraints	L–M/M	Reduce model/data size or rent a short cloud-GPU window.	Up to 5 h setup.	Cloud GPU (€20–50).

3.5 Economic Management

3.5.1 Budget structure

The budget follows the standard software-project cost framework taught in the FIB GEP module [24] and is organised into four components: *human resources by activity*, itemised against the task structure of [Section 3.3](#); *general costs*, covering hardware amortisation, electricity and connectivity overheads, and software-licence costs (zero, since the entire toolchain is open source); *incidentals*, which translate each risk’s contingency action into an expected-value contribution (full cost $\times$ likelihood); and a *contingency reserve* of fifteen per cent of the baseline (human resources plus general costs) for the residual uncertainty that the risk register cannot enumerate explicitly. The wider sustainability framing of this budget—useful life, break-even, and the economic-impact assessment proper—is deferred to [Section 7.2.1](#), which uses the totals derived here as its inputs.

3.5.2 Human resources

Hourly rates are derived from the Spanish ICT salary survey INE-EAES [25] and loaded with a 1.35 multiplier that captures the employer’s social-security contributions in the General Regime of the Spanish social-security system [26]. This multiplier converts the gross annual salary used as a public reference number into the fully loaded annual cost that an employer actually incurs, which is the figure relevant for project budgeting. The hourly rate is then derived from the loaded

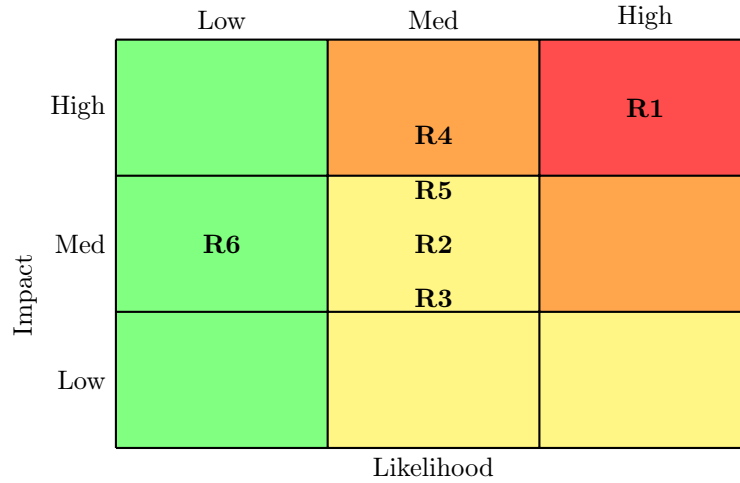


Figure 3.2. Risk-impact matrix locating each of the six register entries on the likelihood/impact plane.

annual cost assuming a working year of 1 760 hours (220 working days, 8 hours per day, after holidays), as prescribed by the FIB GEP economic-management module [24].

Table 3.4. Roles considered for the project and their loaded hourly rates.

Role	Gross (€/year)	Loaded (€/year)	€/h
Project Manager	38 000	51 300	29.15
Research Analyst	33 000	44 550	25.31
ML Engineer	36 000	48 600	27.61
Technical Writer	30 000	40 500	23.01

Since the project is executed by a single person who carries out all four roles, each task in Table 3.2 is costed at the loaded rate of the role that predominantly performs it (for example, T0.5 supervisor meetings and the GEP tasks at the Project Manager rate, the literature survey and error analysis at the Research Analyst rate, the model and pipeline work at the ML Engineer rate, and the thesis-writing tasks at the Technical Writer rate). Summing across roles produces a project-weighted blended rate of €26.75/h on the 509-hour total, which differs slightly from the unweighted mean of the four rates (€26.27/h) because the project’s hour mix is not uniform across roles. The resulting total of €13 615.57 is distributed across work packages as shown in Table 3.5. This figure dominates the budget: roughly 85 % of the project cost is engineering time, which is the expected profile for a research-intensive software project where tooling and hardware are commodity items.

3.5.3 General costs, contingency, and incidentals

General costs (Table 3.6) cover the share of fixed assets and overheads attributable to the 21 weeks of the project: hardware amortisation on the workstation, monitor, and peripherals at a four-year linear rate prorated for the project duration; software licences (zero, since the entire toolchain—Python, PyTorch, OpenCV, LilyPond, PyMuPDF, alumentations, FastAPI, Poetry—is open source); and overheads covering electricity and internet connectivity, derived from the workstation’s average power draw and the Spanish domestic-tariff rate [27] on the assumption that the workstation is dedicated to the thesis for an average of twenty hours per week.

Contingency reserve. A contingency reserve of 15 % is applied over the sum of human resources

Table 3.5. Human-resources budget by work package.

WP	Work package	Hours	Cost (€)
WP0	Project management	130	3 482.50
WP1	Literature review and baseline	78	2 066.20
WP2	Dataset construction	73	1 992.55
WP3	CRNN–CTC development	75	2 075.37
WP4	Iterative refinement	68	1 859.10
WP5	Final evaluation and defence	85	2 139.85
TOTAL human resources		509	13 615.57

and general costs:

$$\text{Contingency} = 0.15 \times (e13\,615.57 + e250.69) = e2\,079.94.$$

The fifteen per cent figure is the project-management default for software projects of this size and follows the guidance of the FIB GEP economic-management module [24]. It is not an estimate of any specific risk but a residual buffer for the unknown-unknown category—the cost overruns that the risk register cannot anticipate by definition.

Incidentals. The incidentals figure of €150.96 (Table 3.7) is the expected-value sum of the four cost-bearing entries of the risk register (Section 3.4). Each entry is the full cost of activating the corresponding contingency action multiplied by the assigned probability of the risk materialising. The total is small in absolute terms because most of the register’s contingency actions consist of reallocating planned effort within the human-resources budget rather than acquiring new resources; only the cloud-GPU rental and the additional data-cleaning cycles incur direct monetary cost.

Table 3.6. General costs.

Item	Cost (€)
Hardware amortisation (desktop, monitor, peripherals)	125.69
Software licences (open-source stack)	0.00
Overheads (electricity + internet)	125.00
TOTAL general costs	250.69

3.5.4 Total budget

3.5.5 Budget control

Budget tracking is performed at each sprint review using three indicators drawn from the earned-value-management framework [28]. The framework requires three quantities per sprint: the *budgeted cost of work scheduled* (BCWS), which is the planned cost of the tasks the sprint should have completed; the *budgeted cost of work performed* (BCWP), which is the planned cost of the tasks actually completed; and the *actual cost of work performed* (ACWP), which is the real cost incurred to complete those tasks. From these three primitives the indicators are derived as follows.

Table 3.7. Incidentals derived from the risk register. Each entry is the cost of activating the corresponding alternative plan, weighted by the risk’s likelihood.

Risk	Contingency action	Full cost (€)	Prob.	Weighted (€)
R1	Additional data cleaning and augmentation cycle	276.10	25 %	69.03
R4	Retraining cycle with simplified vocabulary	276.10	20 %	55.22
R6	Temporary cloud-GPU rental and setup	178.05	15 %	26.71
R5	Scope reduction in extension task	0.00	30 %	0.00
TOTAL incidentals				150.96

Table 3.8. Total project budget.

Budget component	Cost (€)
Human resources	13 615.57
General costs	250.69
Contingency (15 %)	2 079.94
Incidentals	150.96
TOTAL project budget	16 097.16

Cost variance. $CV = BCWP - ACWP$ is the difference between the budgeted cost of completed work and what it actually cost. A positive CV means the sprint came in under budget; a negative CV means it overran. This is the headline cost-health indicator for a single sprint.

Schedule variance. $SV = BCWP - BCWS$ is the difference between the work completed and the work planned, both measured in the common currency of budgeted cost. A positive SV means the sprint completed more than was planned; a negative SV means it fell behind. SV is reported alongside CV because cost and schedule can move in opposite directions—a sprint can come in under cost simply by not completing all its work, which CV alone would not catch.

Cost performance index. $CPI = BCWP/ACWP$ is the dimensionless ratio of value earned to cost incurred. A CPI of 1.0 indicates exact agreement with the plan, below 1.0 indicates an overrun, and above 1.0 indicates an underrun. The advantage of the index form over the raw variance is that it is directly comparable across sprints of different sizes.

Trigger thresholds. Corrective actions are triggered when at least one of the following conditions holds in a sprint review: $|CV| > 10\%$ of the sprint budget, $SV < -5\%$ of the sprint budget, or $CPI < 0.90$ for two consecutive reviews. A single low CPI reading is not by itself a trigger because single sprints have substantial natural variance in a single-person project; sustained underperformance over two sprints is required before the corrective mechanism activates. Once triggered, the corrective action is recorded in the sprint review note and may activate one of the entries of the risk register (Table 3.3)—typically the reallocation of effort from the conditional extension task (T5.3) to the affected work package, or the acceptance of the short cloud-GPU rental from the incidentals budget.

Chapter 4

Design

This chapter describes *what* the system does and *why* each high-level decision was taken. The companion chapter on implementation ([Chapter 5](#)) describes *how* the design is realised in code: the specific libraries, file layout, and engineering choices. The separation follows the principle of presenting the conceptual architecture first—in the vocabulary of music notation and machine learning—so that the reasoning behind each choice is legible before the reader is asked to interpret library names and configuration flags.

4.1 Problem Decomposition

The end-to-end transcription task is decomposed into five sequential stages, with a clearly defined contract between each pair. [Figure 4.1](#) summarises the data flow.

1. **Preprocessing** takes the input bytes (a PDF page or a raster image) and returns a clean, deskewed binary image of the page. This stage is purely image-processing; it is independent of any musical content.
2. **Staff detection** takes the binary page and returns a list of detected *systems*, each consisting of a cropped staff image, the strip of pixels above it (where chord symbols live), and the bounding box on the page. This is the only stage that performs spatial layout analysis; everything downstream operates on one staff at a time.
3. **Melody recognition** takes a single staff image and returns a flat sequence of LMX tokens describing the notes, rests, key, time signature, and barlines printed on that staff. This is the core machine-learning stage.
4. **Grammar correction** takes the raw token sequence and returns a syntactically valid LMX sequence, repairing structural errors (misplaced accidentals, malformed time signatures, runaway measures) without inventing new musical content. This stage is deterministic, not learned.
5. **Chord recognition** takes the chord-strip image associated with each staff and returns a list of jazz chord symbols. It is independent of the melody recognition stage and uses a separate neural network.

Why a five-stage decomposition. Three constraints drove this decomposition. First, the chord-symbol stream and the melody stream require fundamentally different recognisers—one is staff-relative pattern recognition, the other is text-like OCR—so they must be separable. Second, the grammar of the LMX output is more easily enforced by a deterministic validator than learned end-to-end from a finite training set; isolating that logic in a dedicated stage produces more reliable output without enlarging the model. Third, both major surveys of the field [1], [2]

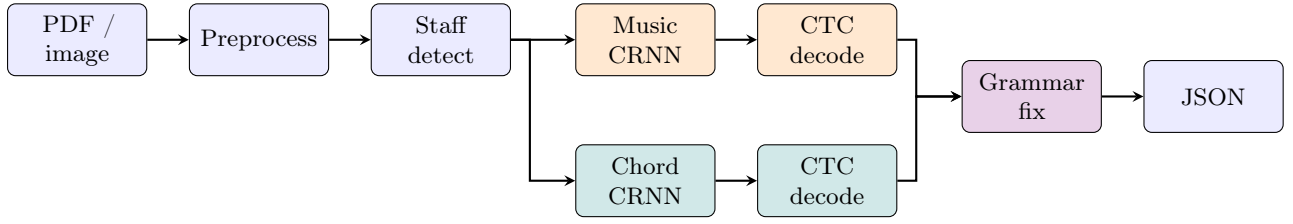


Figure 4.1. High-level data flow of the five-stage inference pipeline. A preprocessing front-end normalises the input page; a staff detector emits a staff image and a chord-strip image for every detected system; two independent CRNN–CTC recognisers process the two streams in parallel; a deterministic grammar fixer repairs structural defects in the music stream; and the merged result is serialised as JSON. Each block has a single responsibility and a typed input/output contract.

explicitly recommend modular decomposition for OMR systems that target specific notation styles, because it allows each module to be replaced or retrained independently as the target domain evolves.

4.2 Data Strategy

The absence of a labelled corpus of jazz lead sheets is the single most important constraint on this project. The data strategy is therefore designed around two complementary ideas: *synthetic supervision* for breadth and *domain augmentation* for realism.

4.2.1 Synthetic supervision from PrIMuS

PrIMuS [6] provides 87 678 monophonic music incipits, each accompanied by ground-truth symbolic annotations (clef, key, time signature, and the sequence of notes and rests on the staff). The corpus is, however, distributed with renderings in the Sibelius engraving style, which does not match the Real Book aesthetic that the system must face at inference time. The chosen strategy is to keep the *semantic content* of every PrIMuS sample but to discard the supplied renderings and re-render each sample in the LilyJAZZ engraving style. This yields a corpus that is large enough to train a deep model, free of manual annotation cost, and visually aligned with the target domain.

Per-sample rendering variations sample the natural visual variability of real lead sheets without altering the musical content: the time signature is rendered numerically or as the C / cut-time glyph with equal probability; cautionary accidentals are added in roughly half of samples; the global staff size is drawn uniformly from a six-element range, varying note size and horizontal density; and unbeamed eighth notes appear in a small fraction of samples to expose the model to both beamed and unbeamed groupings.

4.2.2 Domain augmentation: from clean renders to scanned pages

Even after the engraving style is corrected, a model trained only on clean LilyJAZZ renders generalises poorly to real scans: the feature distribution shifts substantially when crisp black symbols on a white background are replaced by blurred, uneven, noisy images with variable contrast and geometric distortion. The system therefore generates a second, augmented version of every training image that simulates the distortions characteristic of real-world scans: page warping and lens distortion, scan misalignment, focus blur, exposure variation, sensor noise, uneven illumination, ink bleed, halftone banding, and JPEG compression. The model is trained on the union of clean and augmented samples, with the augmented copies sharing the same ground-truth label. This is the same recipe followed by every recent end-to-end OMR system [11], [18], [19], [20]; the specific list of distortions used here is calibrated against a sample of real Real Book scans.

4.2.3 Domain restriction: filtering the training set

PrIMuS contains many samples that lie outside the lead-sheet domain: multi-staff renders, non-treble clefs, exotic time signatures. Three deliberate filters are applied at dataset construction time to restrict training to the in-scope distribution: multi-staff samples are dropped, samples with non-treble clefs are dropped, and samples with time signatures outside the eight-element jazz common-time set (4/4, 3/4, 2/4, 2/2, 6/8, 6/4, 5/4, 12/8) are dropped. This is a deliberate *contract*, not a workaround: disabling the filters would force the model to spend capacity on visual patterns that never appear at inference, without improving its performance on the target distribution. The same contract is enforced by the grammar fixer at inference time, which rewrites any predicted clef other than G2 to G2 and rejects time signatures outside the allowed set.

4.2.4 Fine-tuning on real pages

The synthetic-to-real gap cannot be fully closed by augmentation alone: the visual distribution of real Real Book scans has detail—specific ink-pen artefacts, paper grain, page-layout idiosyncrasies—that no practical augmentation pipeline can reproduce exactly. The design therefore plans for a second training phase: after the synthetic model has converged, a small set of manually annotated Real Book pages is injected into the training split (only) and the model is fine-tuned on the combined corpus. The fine-tuning stage is intentionally lightweight; its purpose is to adjust the feature distribution rather than to teach the model new musical concepts.

4.3 Output Representation: LMX

The model’s output is a flat sequence of LMX tokens. This section defines the LMX grammar used in this thesis, describes the vocabulary the model learns, and justifies the design choices made.

4.3.1 Grammar

A valid LMX sequence is a sequence of measures, each introduced by a `measure` token and optionally followed by a clef, key, or time declaration before its musical content. Within a measure, the content is a flat sequence of notes and rests; no nesting or polyphony is permitted.

```

SEQUENCE := HEADER ELEMENT* (BARLINE ELEMENT*)*
HEADER   := "measure" [CLEF] [KEY] [TIME]
BARLINE  := "measure" [KEY] [TIME]
CLEF     := "clef:G2"
KEY      := "key:fifths:" <fifths>
TIME     := "time" "beats:" <n> "beat-type:" <d>
ELEMENT  := NOTE | REST
NOTE     := "pitch:" <A-G> "octave:" <0-8> DURATION DOT*
          [ACCIDENTAL] [TIE_STOP] [FERMATA]
REST     := "rest" DURATION DOT* [FERMATA]
DURATION := "whole"|"half"|"quarter"|"eighth"|"16th"|"32nd"|"64th"
ACCIDENTAL := "flat" | "sharp" | "natural"
TIE      := "tied:start" ... "tied:stop"

```

Three encoding choices in this grammar deserve specific justification.

Three-token time signatures. The time signature is emitted as three separate tokens (`time`, `beats:n`, `beat-type:d`) rather than a single joint token `time:n/d`. This factorisation reduces the vocabulary size from forty-plus possible combinations to roughly a dozen `beats:` and `beat-type:` values, and it lets the model share representations between related time signatures (4/4 and 2/4 share `beat-type:4`; 6/8 and 12/8 share `beat-type:8`).

Bare accidental tokens. Accidentals appear as bare words (`flat`, `sharp`, `natural`) rather than prefixed (`acc:flat`). This matches the OLIMPIC LMX convention [11] and keeps the token sequence compact. Accidentals appear *after* the duration in the token stream because that mirrors the temporal order in which a reader processes a note: pitch position, then duration, then any required display marks.

Display-only accidentals. An accidental token (`flat`, `sharp`, `natural`) signals that a flat, sharp, or natural sign should be *drawn* on the staff; it does not encode the pitch’s chromatic alteration. The chromatic pitch is already determined by the pitch letter, the octave, and the active key signature. This is the rule used by MusicXML, and it makes the model’s prediction task more local: deciding whether to display an accidental depends only on the current measure and the key signature, not on a global pitch inference. A consequence is that the converter must compute—per note—whether the pitch’s alteration conflicts with the active key signature or with a prior alteration in the same measure, and emit an accidental token only when a sign should be drawn.

4.3.2 Vocabulary

The model treats LMX tokens as opaque class labels. Three special tokens are reserved by convention: index 0 is the CTC blank, index 1 is the padding token used during batch collation, and index 2 is the out-of-vocabulary token. The remaining indices, sorted alphabetically, encode the LMX tokens that appear in the training corpus.

The vocabulary covers seven categories: pitch letters (`pitch:A` through `pitch:G`); octave numbers (`octave:0` through `octave:8`); the eight duration words; the dot token; the three accidentals; the `measure` and `rest` structural tokens; the tie tokens; the clef token `clef:G2`; key tokens `key:fifths:-6` through `key:fifths:7`; and the time-signature tokens. The full pitch and octave ranges are always reserved even if absent in the corpus, so that out-of-distribution pitch predictions decode gracefully rather than collapsing to `<unk>`.

4.3.3 Why LMX over alternatives

Section 2.5 surveyed the available symbolic output formats; this thesis adopts LMX rather than MusicXML directly, the PrIMuS native formats, or ****kern**. LMX is round-trippable to MusicXML—the format downstream music applications expect—without requiring the model to emit verbose XML tags; it is the same representation used by the OLIMPIC reference work [11], keeping the present results directly comparable; and its flat, whitespace-separated form is directly compatible with CTC training, with no linearisation logic required at the model boundary.

4.4 Model Design

The melody recognition stage is a Convolutional Recurrent Neural Network trained with the Connectionist Temporal Classification loss (CRNN–CTC). This section justifies that architectural choice and its principal hyperparameters in conceptual terms. The implementation details—PyTorch module structure, exact stride table, optimiser state—live in Chapter 5.

4.4.1 Why CRNN–CTC

A single staff from a jazz lead sheet is read strictly left to right and produces a variable-length sequence of tokens that is shorter than the image is wide. This matches the data assumptions of CTC [29] exactly: the loss learns the alignment between an input feature sequence and a shorter output token sequence without per-frame supervision, and it tolerates the two-dimensional context (relative vertical position on the staff) needed to disambiguate pitch from notehead shape. End-to-end CRNN–CTC has been the dominant architecture for monophonic OMR since

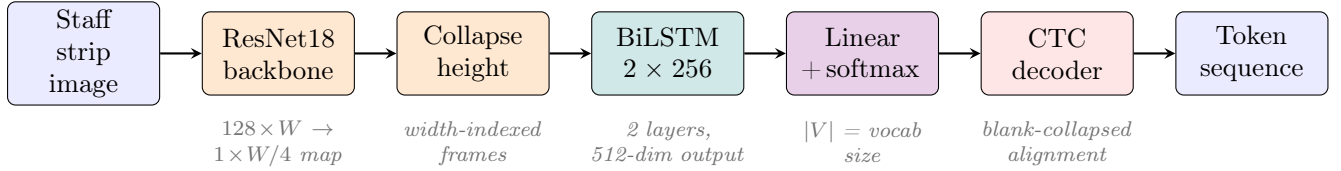


Figure 4.2. Block diagram of the CRNN-CTC melody recogniser. A ResNet18 backbone with asymmetric strides compresses the input staff strip from $128 \times W$ down to a $1 \times W/4$ feature map; the height-collapsed sequence of feature columns is consumed left-to-right by a two-layer bidirectional LSTM (256 hidden units per direction, 512-dim concatenated output); a linear projection produces a per-frame distribution over the vocabulary; and CTC decoding collapses the blank-padded alignment into a flat LMX token sequence.

Calvo-Zaragoza and Rizo [17] and remains the strongest baseline for the present domain [11], [18]; the full literature trail behind this choice is laid out in [Section 2.2](#). A convolutional backbone provides translation-invariant feature extraction along the staff width; a bidirectional recurrent layer captures the local dependencies needed to interpret accidentals, dots, and beam groupings; and CTC blank-collapsing decode produces a clean symbolic output. A transformer of comparable capacity would in principle perform at least as well but at substantially higher data and compute costs that are not justified for monophonic input ([Section 2.3](#)).

4.4.2 Backbone choice

The default backbone is a modified ResNet18 [30] with asymmetric strides that compress the height dimension aggressively while preserving width, leaving the horizontal dimension as the CTC time axis. A five-block VGG-style stack is retained as a legacy alternative for backward compatibility with earlier checkpoints; it converges more slowly and is not used for new experiments. ResNet18 was preferred for three reasons: its residual connections give the gradient a direct path through the network and accelerate convergence; its pre-trained weights from ImageNet provide a reasonable initialisation for the first convolutional layers, even though the downstream task is monochrome rather than colour; and its parameter count fits comfortably within the VRAM budget of a single consumer GPU (see [Chapter 3](#)).

4.4.3 Temporal resolution and width compression

The backbone compresses the width of the input image by a factor of four, leaving the model with one feature vector per four input columns of the original image. This factor was chosen by a trade-off: a smaller compression ratio preserves more time steps and so admits longer output sequences without saturating CTC alignment, but increases compute cost; a larger ratio risks producing more output tokens than time steps, which CTC’s `zero_infinity` option silently masks but which corrupts gradient estimates. At a factor of four, all in-scope samples comfortably satisfy the CTC length constraint, and the time axis remains rich enough to disambiguate adjacent notes even on densely-engraved staves.

4.4.4 Recurrent layer

The convolutional features are fed to a stacked bidirectional LSTM [31]. Two layers are used, with 256 hidden units per direction; this gives the model bidirectional context over the entire staff while keeping parameter count modest. A single-layer LSTM was found to under-fit on dense passages (see ablations in [Chapter 6](#)); a three-layer LSTM gave no measurable gain over two layers and slowed training. Dropout between LSTM layers, applied during training only, regularises the model against the inevitable overfitting that arises from training on a synthetic corpus whose support is narrower than the inference-time distribution.

4.5 Two-Stream Architecture for Lead Sheets

A jazz lead sheet contains two parallel visual streams: the melody on the staff and the chord symbols typeset above it. Their visual nature is fundamentally different. The melody is a sequence of graphical symbols positioned relative to the staff lines; correct interpretation depends on the clef and key signature. The chord symbols are short alphanumeric strings (Cmaj7, F#m7b5, Bb13); their interpretation is a text-recognition task and does not depend on staff geometry.

The architectural response is a *two-stream* design: two independent neural networks, each specialised to its own visual distribution, with results merged in post-processing. The melody network is the CRNN–CTC described in Section 4.4; its vocabulary is the LMX token set. The chord network is a separate CRNN–CTC of the same family but character-level: its vocabulary is the 26-character alphabet sufficient to spell every chord symbol in the Real Book.

4.5.1 Why two networks rather than one

A single network trained to emit both LMX tokens and chord characters would have to learn two unrelated visual distributions and a vocabulary that is the union of their token sets, with most tokens absent from any given input region. The two-stream design instead lets each network specialise: its training data is purely in-domain, its vocabulary is minimal, and its training schedule can be set independently. Both surveys of the field [1], [2] explicitly recommend this decomposition for systems that handle text-like adjuncts to staff notation.

4.5.2 Independent training, joint inference

The two networks are trained on disjoint datasets and produce independent outputs. At inference time, the staff detection stage emits both a staff image and a chord-strip image for each detected system; the two are processed in parallel and their outputs are joined into a single JSON segment keyed by the system’s bounding box on the page. No cross-attention or shared representation links the two networks; this is a deliberate simplicity choice.

4.5.3 Chord vocabulary and notation conventions

Chord symbols in the Real Book follow a stylised convention that differs from many notation packages: minor chords are written with a hyphen (G-7 rather than Gm7), major-seventh chords with maj (Fmaj7), half-diminished with the ø glyph (Bø), and diminished with the spelled-out dim suffix (Cdim7). The character vocabulary covers the seven root letters, two accidentals, four quality letters that spell maj, dim, and sus, the symbols -, +, ø for minor, augmented, and half-diminished, the digits used in extensions (1, 3, 6, 7, 9), and the separator characters (space and slash). Out-of-scope chord features—alt, double accidentals, parenthesised alterations—are deliberately absent from the vocabulary and are mapped to nearest in-vocabulary spellings during label canonicalisation.

4.6 Training Strategy

4.6.1 Two-phase plan

The training procedure has two distinct phases. *Phase 1* trains the model from scratch on the synthetic LilyJAZZ corpus, with both clean and augmented versions of every sample included in each epoch. *Phase 2* fine-tunes the resulting checkpoint on a small manually-annotated set of Real Book pages, injected into the training split only. The validation and test splits remain drawn exclusively from the synthetic corpus throughout, so that phase-2 fine-tuning does not contaminate the held-out evaluation.

This is a textbook synthetic-pretrain → real-fine-tune transfer learning plan; its effectiveness is the principal experimental question of [Chapter 6](#).

4.6.2 Loss and supervision signal

The supervision signal is the LMX token sequence associated with each image. CTC loss takes the log-probability tensor (one vector per output time step), the target token sequence, and their respective lengths, and computes the negative log-likelihood of the target marginalised over all valid alignments to the input time axis. The CTC blank class (index 0) absorbs time steps where no symbol is emitted, which is the mechanism by which the model handles variable-rate output without explicit alignment.

4.6.3 Data splits

The corpus is partitioned deterministically into train, validation, and test splits with a fixed seed. Validation and test fractions are each 10 %; the remaining 80 % is used for training. The same sample identifier always falls into the same split across runs, so successive experiments are directly comparable. When fine-tuning data is added, it is injected only into the train split; no fine-tuning sample ever enters validation or test.

4.6.4 Header-stripping augmentation

Real Book pages often span multiple staves per system; every staff after the first omits the clef, key, and time-signature header. To prevent the model from over-relying on the header, a fraction of training samples (40 %) have their header pixels cropped out and the corresponding tokens removed from the label. This forces the model to learn pitch and rhythm in the absence of an explicit clef, which it can do because the staff geometry alone is sufficient given the treble-clef-only contract.

4.6.5 Rare-token oversampling

Two LMX tokens (`tied:start` and `tied:stop`) are visually subtle and statistically rare in the corpus, so the model under-predicts them. The training loader duplicates—in the training split only—every sample whose label contains a rare token, with a configurable multiplier. This is a class-imbalance fix applied at the sample level rather than in the loss function, which CTC does not easily admit.

4.7 Grammar Correction as a Design Layer

The CRNN model is trained to maximise the probability of the correct token sequence; nothing in the training objective forces its output to be a syntactically valid LMX sequence. On most samples it is, but on dense or degraded inputs the raw prediction can contain malformed fragments: a `pitch:C` not followed by an `octave:` token, an accidental before the duration of its note, a barline that arrives before the accumulated beats fill the current measure.

A dedicated *grammar fixer* sits between the model and the JSON output. Its responsibilities are:

- enforce the LMX grammar laid out in [Section 4.3](#) (correct token order within a note, correct duration after `rest`, correct nesting of tied groups);
- normalise predictions outside the lead-sheet contract: any clef other than `clef:G2` is rewritten to `clef:G2`; time signatures outside the allowed eight-element set are rejected;

- clamp predicted octaves to the in-domain range $[3, 7]$, preventing rare misreads from producing unreachable pitches;
- count cumulative beats per measure against the active time signature and insert implicit barlines when the count overflows, producing well-formed measure boundaries even when the model misses a printed barline.

The grammar fixer is purely deterministic: it never invents new musical content, only repairs structural defects in the model’s output. A learned alternative—training the model end-to-end against a grammar-aware loss—is conceivable but adds complexity and offers limited gain for a vocabulary as small and as well-structured as LMX. Keeping grammar enforcement out of the loss also keeps the model’s output legible: every token in the raw prediction is interpretable in the same way as the corresponding ground-truth token.

4.8 Evaluation Design

4.8.1 Primary metric: Symbol Error Rate

The primary metric is the Symbol Error Rate [9]: the Levenshtein edit distance between the predicted token sequence and the ground truth, normalised by the length of the ground truth. SER is reported on both the clean and the scanned versions of the test split, so that the contribution of the augmentation pipeline is visible.

4.8.2 Category-level error breakdown

SER aggregates errors of very different musical significance: a missed barline counts identically to a wrong pitch. To make this distribution legible, the evaluation reports a category-level breakdown of edits attributable to each LMX token family: **measure** tokens, tie tokens, pitch letters, octaves, durations, accidentals, and key/time-signature tokens. This is the same diagnostic information that OMR-NED provides as a single number [23], expressed here as a table rather than committed to a specific normalisation.

4.8.3 Melodic SER

A complementary *melodic* SER is also reported, computed on the same edit distance after stripping **measure** and tie tokens. Structural tokens dominate the absolute edit count on the current corpus but are largely musically irrelevant once a downstream renderer re-bars the output; the melodic SER captures how often the model gets the *actual notes* right.

4.8.4 Domain-gap evaluation

The principal experimental question of this thesis is whether synthetic pretraining plus fine-tuning closes the gap between training on rendered images and inference on real scans. Two ablations support this evaluation: (i) the model trained only on clean LilyJAZZ renders is evaluated on real Real Book pages, exposing the raw synthetic-to-real gap; (ii) the same model after fine-tuning on a small annotated real set is evaluated on the same pages, quantifying the reduction in the gap. The fine-tuning protocol is the same as that described in [Section 4.6](#).

Chapter 5

Implementation

This chapter describes how the design presented in [Chapter 4](#) is realised in code: the libraries chosen at each layer, the structure of the project repository, the concrete configuration values used, and the engineering decisions taken to keep the system reproducible and within the compute budget of a single consumer GPU. The chapter is deliberately separate from the design chapter so that the conceptual architecture can be read independently of any particular library or framework choice.

5.1 Technology Stack

Table [5.1](#) lists the principal libraries used in each layer of the system. The selection criterion in every case was the smallest, most widely-used library compatible with the design constraints; no component was chosen for novelty.

Layer	Library	Role
Deep learning	PyTorch 2.10+	Model, loss, optimiser, mixed-precision training.
CNN backbone	<code>torchvision</code>	ResNet18 weights and module definition.
Image processing	OpenCV	Binarisation, deskewing, morphological staff detection.
PDF rasterisation	PyMuPDF (<code>fitz</code>)	Render PDF pages to grayscale arrays at configurable DPI.
Music rendering	LilyPond + LilyJAZZ	Generate training PNGs from PRIMuS semantic files.
Data augmentation	<code>albumations</code>	Scan-simulation transform pipeline.
Numerics	NumPy	Tensor / array interop.
Web service	FastAPI + Uvicorn	REST API exposing the inference pipeline.
Package manager	Poetry	Dependency isolation, deterministic environment.

Table 5.1. Principal libraries used in each layer of the system. Versions are pinned in `pyproject.toml`; Poetry produces a reproducible virtual environment across machines.

The runtime target is a single NVIDIA RTX 3060 with 12 GB of VRAM, which is the hardware available to the author for the duration of the thesis. All architectural choices in [Chapter 4](#) were taken with this budget in mind; the implementation does not require multi-GPU support, distributed training, or specialised cloud infrastructure.

5.2 Repository Structure

The project source tree is organised by responsibility rather than by file type.

src/	
cli.py	unified CLI (10 subcommands)
CRNN CTC/	model, training, vocab, evaluation
model.py	CRNN-CTC architecture
vocab.py	Vocabulary with fixed blank/pad/unk
dataset.py, config.py	PrIMuS dataset + Config dataclass
train.py, evaluate.py	training loop and evaluation
chord_dataset.py	synthetic chord-strip dataset
chord_train.py	chord CRNN training from scratch
chord_finetune.py	fine-tune chord CRNN on real strips
data_processing/	dataset generation and augmentation
generate_realbook.py	PrIMuS -> LilyJAZZ render
semantic_to_lmx.py	PrIMuS .semantic -> LMX label
augment_scanned.py	scan-simulation augmentation
chord_render.py	synthetic chord-strip renderer
generate_chord_crops.py	bulk chord-strip generation
extract_real_chord_strips.py	pre-label strips from PDFs
omr_pipeline/	inference pipeline
pipeline.py	run_pipeline() entry point
preprocess.py	load / binarise / deskew
staff_detect.py	morphology -> 5-line clusters
inference.py	CRNN-CTC music recognition
chord_recognizer.py	chord OCR
chord_postprocess.py	jazz-chord grammar filter
grammar_fix.py	LMX token sequence validator
api/	FastAPI web server

Three structural principles are enforced. First, all music-notation mapping tables (clef IDs, key-signature encodings, duration tokens) live in a single module (`CRNN CTC/lilypond_render.py`) and are imported by every other module that needs them; this prevents the rendering pipeline and the label converter from drifting out of sync. Second, the inference pipeline (`omr_pipeline/`) has no dependencies on the training code beyond the `Vocabulary` and `CRNN` classes; it can be deployed without the training tree. Third, the CLI module (`cli.py`) is the only entry point: every multi-step operation (data generation, training, evaluation, vocab building) is exposed as a CLI subcommand, with the same flags and defaults across operations.

5.3 Data Pipeline Implementation

The data pipeline is implemented as four CLI subcommands run in sequence: **render**, **convert**, **augment**, and **vocab**. A wrapper subcommand (**pipeline**) runs all four with a shared set of paths.

Every stage uses the Python `multiprocessing` module for CPU parallelism, with the worker pool size set by a CLI flag (default 8). The Python global interpreter lock makes threading useless for these CPU-bound stages.

5.3.1 Render: PrIMuS to LilyJAZZ

The **render** subcommand walks the PrIMuS package directories, parses each `.semantic` file, normalises the clef to G2 if needed (preserving absolute pitches by writing them at the equivalent treble-clef position), constructs a LilyPond source file, and invokes the `lilypond` subprocess at 200 DPI. The output PNG is auto-cropped to remove white margins.

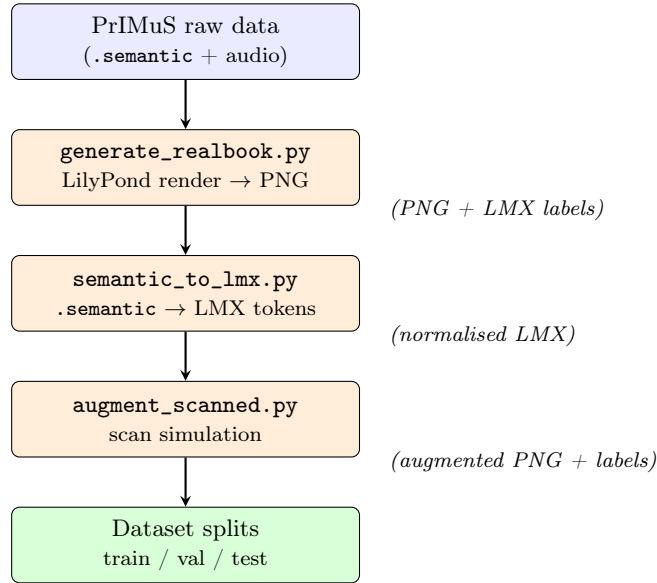


Figure 5.1. Data pipeline: PrIMuS semantic files are re-rendered in LilyJAZZ, converted to LMX labels, augmented to simulate scanned conditions, and indexed into a vocabulary file.

The LilyJAZZ font stylesheet is not bundled with LilyPond and must be installed manually into the LilyPond `share` directory before rendering can begin; the installation script is documented in `docs/data_pipeline.md` and the project assumes its presence at runtime.

5.3.2 Convert: PrIMuS .semantic to LMX

The `convert` subcommand parses each `.semantic` file token-by-token, computes the display-accidental rules described in Section 4.3 from the pitch spelling and the active key signature, and emits an LMX label file co-located with the rendered PNG. The conversion is direct: no intermediate MusicXML is produced. A non-obvious aspect of the implementation is the *implicit-key fix*: PrIMuS omits the `keySignature-` token entirely for C major samples, but the model must learn to predict `key:fifths:0` whenever the staff has no accidentals. The converter detects this case and injects the C-major key token before the time signature, ensuring that every training sample has an explicit key label.

5.3.3 Augment: scan simulation

The `augment` subcommand applies the `albumentations` [32] transform pipeline to every clean image and writes the result to a parallel directory. The transforms are listed in Table 5.2; their probabilities and parameter ranges were tuned against a sample of real Real Book scans. After the `albumentations` pipeline, six additional non-library operations are applied in order, each with an independent probability: morphological ink dilation (0/1/2 erosion iterations chosen stochastically), staff-line thickness variation ($p = 0.40$), paper-tone remapping from pure white/black to a paper-like grey range, a radial vignette ($p = 0.55$), a directional uneven-illumination gradient simulating a book-spine shadow ($p = 0.45$), and faint horizontal halftone banding ($p = 0.25$).

A second, lighter augmentation layer is applied online during training inside the `Dataset.__getitem__` method: per-sample brightness and contrast jitter, low-amplitude noise, and a small horizontal translation, each with probability 0.5. Without this online jitter the model overfits to the exact pixel realisations that the offline pipeline produced. The online cost is approximately 50 μ s per sample, small enough to be absorbed by the data loader’s worker processes.

Transform	Probability	Range / purpose
ElasticTransform	0.82	page warping, staff curvature
GridDistortion	0.72	grid bending
OpticalDistortion	0.38	lens distortion
Affine	0.85	rotation $\pm 4^\circ$, shear $\pm 1.2^\circ$, scale 0.97–1.03
GaussianBlur	0.62	$\sigma \in [0.2, 1.1]$
Sharpen	0.48	oversharpened-scanner simulation
RandomToneCurve	0.72	exposure variation
GaussNoise	0.78	sensor noise
RandomBrightnessContrast	0.70	brightness ± 0.06 , contrast 0.03–0.18
ImageCompression	0.35	JPEG quality 72–92

Table 5.2. The augmentations transform pipeline applied during the augment stage. Probabilities and parameter ranges are calibrated against a sample of real Real Book scans.

[TODO: FIGURE: Six side-by-side staff strips. Left: clean LilyPond render. Then five progressively augmented versions: (1) Gaussian blur ($\sigma = 1.5$), (2) elastic warp, (3) ink-bleed simulation, (4) JPEG compression artifacts, (5) all combined. Generate with: poetry run python scripts/generate_augment_samples.py -n-samples 1 -show-steps. Target: 6 panels, width=\textwidth, height approx 2 cm each.]

Figure 5.2. One clean LilyJAZZ render and five scan-simulated copies produced by the augment stage.

5.3.4 Vocab: build the token index

The `vocab` subcommand scans every `.lmx` file in the training corpus, collects the set of unique tokens, sorts them alphabetically, and writes them to a plain-text file with one token per line. The three special tokens (blank, pad, unk) are not written to the file; they are injected at indices 0, 1, and 2 by the `Vocabulary` class at load time. The full pitch and octave ranges are reserved in the vocabulary even if some entries are absent from the corpus, so that the model’s softmax output has a stable shape across data-pipeline reruns.

5.4 Model Implementation

5.4.1 Module structure

The model is implemented as three PyTorch `nn.Module` subclasses: a CNN backbone, a recurrent stack, and a thin linear projection head. The `CRNN` class composes them and exposes a single `forward(images, image_widths)` method that returns log-softmax probabilities ready for the CTC loss.

5.4.2 ResNet18 backbone modifications

The torchvision ResNet18 is used as a starting point but with two critical modifications. First, the first convolutional layer is replaced to accept single-channel input rather than three-channel RGB; pre-trained weights for this layer are recomputed by averaging the RGB weights along the input channel axis. Second, the strides of three convolutional stages are made asymmetric so that the height dimension is compressed eight times more aggressively than the width dimension:

```
conv1      stride (2, 1)
maxpool    stride (2, 2)
layer1     stride (1, 1)
layer2     stride (2, 2)
layer3     stride (2, 1)
layer4     stride (2, 1)
AdaptiveAvgPool2d -> output height 1
```

The cumulative effect is an input of shape $(B, 1, 128, W)$ producing an output of shape $(B, 512, 1, W/4)$. The height dimension is squeezed before the recurrent stack so the input to the BiLSTM is $(W/4, B, 512)$. A consequence of these asymmetric strides is that the time axis (width) is preserved with only a $4\times$ compression while the height collapses to a single feature vector; this is exactly what the CTC time axis needs.

5.4.3 Bidirectional LSTM

The recurrent stack is a two-layer bidirectional `nn.LSTM` with 256 hidden units per direction, dropout of 0.3 between layers, and the standard PyTorch packed-sequence handling for variable-width batches. The output is projected to the vocabulary size by an `nn.Linear` followed by `log_softmax` along the vocabulary dimension; this is the input shape required by `nn.CTCLoss`.

5.4.4 CTC loss configuration

The CTC loss is constructed with `blank=0` and `zero_infinity=True`. The latter is essential: it silently zeroes the loss contribution of samples whose label length exceeds their effective input length ($W/4$), which would otherwise cause a NaN-inducing infinity. On the in-scope corpus this masking is virtually never triggered because the multi-staff filter ([Section 4.2](#)) removes the samples most at risk; `zero_infinity` is a safety net rather than a routine code path.

5.4.5 Training loop

The training loop is a single-process, single-GPU implementation built on PyTorch’s standard primitives. Each epoch performs the following:

1. Iterate over the training `DataLoader` with `num_workers=10`; each batch is a dictionary of right-padded image tensor, packed label tensor, label lengths, and image widths.
2. Forward pass through the CRNN inside an `autocast` context for mixed-precision training; CTC loss is computed in single precision.
3. Backward pass via the `GradScaler`’s scaled-loss path; gradient norm clipped at 5.0 to stabilise the LSTM.
4. Optimiser step (`AdamW` [33], peak learning rate 1×10^{-3} , weight decay 1×10^{-4}).
5. Scheduler step (`OneCycleLR` [34], single-cycle cosine annealing, warm-up fraction 0.08).
6. Validation pass at end of epoch: greedy CTC decode on the full validation set, SER computed against ground truth, best checkpoint saved if SER improved.

7. Append the epoch’s metrics to `training_log.csv` in the model directory.
8. Stop if SER has not improved for 12 consecutive epochs (early stopping).

5.4.6 Hyperparameter summary

Table 5.3 lists every hyperparameter used by the default training configuration. All defaults are encoded in the `Config` dataclass and serialised to `config.json` at training start.

Hyperparameter	Default	Notes
<code>img_height</code>	128	fixed; image is resized to this height
<code>max_image_width</code>	2048	wider images are clamped
<code>backbone</code>	<code>resnet18</code>	VGG retained for legacy checkpoints
<code>rnn_hidden</code>	256	per direction
<code>rnn_layers</code>	2	BiLSTM depth
<code>dropout</code>	0.3	between LSTM layers
<code>batch_size</code>	16	fits in 12 GB at <code>img_height=128</code>
<code>lr</code>	1×10^{-3}	OneCycleLR peak
<code>weight_decay</code>	1×10^{-4}	AdamW
<code>warmup_frac</code>	0.08	of total steps
<code>max_grad_norm</code>	5.0	gradient clipping
<code>epochs</code>	60	maximum, with early stopping
<code>early_stopping_patience</code>	12	epochs without val SER improvement
<code>val_frac</code>	0.10	validation split fraction
<code>test_frac</code>	0.10	test split fraction
<code>strip_header_prob</code>	0.40	header-strip augmentation rate (training only)
<code>online_aug_prob</code>	0.50	in-loader jitter rate (training only)
<code>rare_lmx_oversample</code>	2	multiplier for samples with tie tokens
<code>num_workers</code>	10	<code>DataLoader</code> worker processes

Table 5.3. Default hyperparameters of the `Config` dataclass. Every value is overridable via a CLI flag; the entire `Config` is serialised alongside the checkpoint so that evaluation always uses the same settings as training.

5.5 Chord OCR Implementation

The chord recognition network reuses the CRNN–CTC machinery of the melody network but is trained on a different data stream with a different vocabulary.

5.5.1 Synthetic chord-strip generation

The `chord_render` module defines the canonical set of chord qualities (major, minor, dominant 7, major 7, half-diminished, diminished, augmented, suspended) with sampling weights derived from

the empirical frequency of each quality in the Real Book. The `generate_chord_crops` script renders strips of one to five chords each, in the LilyJAZZ font, at the same DPI as the staff renders, then applies an `albumentations` pipeline tuned for the visual conditions of the chord strip (smaller scale, less geometric distortion, more colour-channel jitter) and an additional “Real Book clutter” overlay that adds artefacts such as binder-hole shadows, staff-line bleed-through from the staff below the chord strip, and occasional slash-repeat marks.

Twelve thousand training strips and twelve hundred validation strips are generated for the default configuration; the chord training script consumes them via a CSV manifest.

5.5.2 Chord network architecture

The chord network is the same ResNet18 + BiLSTM + linear-projection stack as the melody network, but with a 29-class output (26 character tokens plus blank, pad, unk). The input image is resized to 64 pixels in height (half the staff height) before being fed to the network; this is the height to which the chord strip is resampled at inference time as well.

5.5.3 Fine-tuning on hand-labelled strips

The synthetic chord network is fine-tuned on a small set of real chord strips extracted from Real Book pages and manually labelled. Extraction reuses the staff-detection module of the inference pipeline: chord strips are cropped from the region directly above each detected staff, and the synthetic chord model is run on each strip to provide a first-pass label that the human reviewer corrects rather than typing from scratch. Labels are stored in a JSON Lines file with a `status` field that allows the labeller to skip strips that contain no chords or are too degraded to read.

Fine-tuning mixes real and synthetic strips with a weighted sampler: real strips carry weight 1.0, synthetic strips weight 0.4. The synthetic admixture prevents catastrophic forgetting of rare chord qualities under-represented in the small real set. Labels go through a *canonicalisation* step before the vocabulary filter: `m7b5` is rewritten to the $\emptyset$ glyph, `Cm7` to `C-7`, and characters not in the chord vocabulary are dropped silently.

5.6 Inference Pipeline Implementation

The inference pipeline is a single function, `run_pipeline`, that accepts the raw bytes of a PDF or image plus the original filename and returns a structured JSON dictionary. It is invoked by the FastAPI endpoint described in [Section 5.8](#); no separate CLI subcommand is exposed for single-page inference because the typical use-case is uploading a page through the web UI.

5.6.1 Preprocessing

PDFs are rasterised with PyMuPDF at the DPI specified by `OMR_PDF_DPI` (default 300, clamped to [72, 600]). The current implementation processes only page 0 of multi-page PDFs; multi-page batching is left to future work. Raster images are loaded directly via OpenCV. Binarisation uses Contrast Limited Adaptive Histogram Equalisation (CLAHE) followed by Otsu thresholding; this combination is more robust to uneven scanner illumination than plain Otsu on the raw image. Deskewing tests a range of rotation angles and selects the angle that maximises the variance of the horizontal projection profile (sharper row peaks correspond to better-aligned staves).

5.6.2 Staff detection

Staff detection is a classical morphological pipeline: a wide horizontal opening kernel extracts staff-line candidates, row centroids are computed and merged across closely-spaced rows to handle thick lines, and the centroids are clustered into groups of five consecutive nearly-equispaced lines.

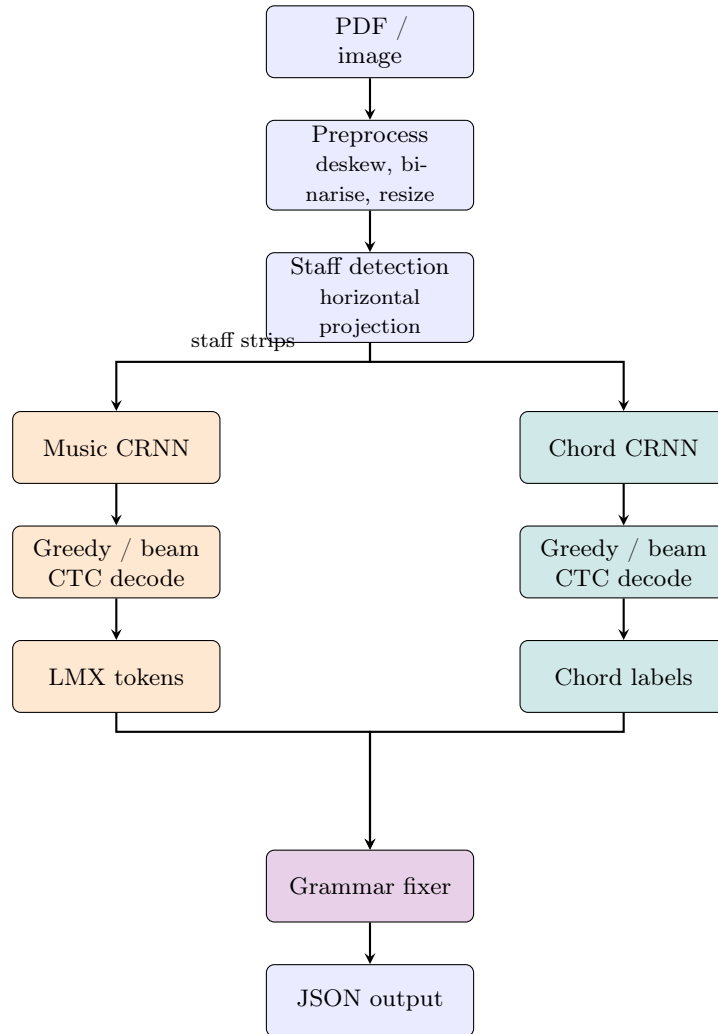


Figure 5.3. Two-stream inference flow. Music recognition and chord recognition run as independent CRNN–CTC streams; their outputs are joined per staff and validated by the grammar fixer before serialisation as JSON.

For each cluster, the system emits a **System** record containing the staff image, the strip of pixels above it (where chord symbols live, vertical span computed as the gap to the previous staff), and the bounding box on the page.

This stage is deliberately not a neural network: classical morphology is fast, deterministic, and accurate enough on the well-formatted Real Book input. A future revision could replace it with a learned segmenter, but the current implementation has not been the bottleneck of any failure mode observed during evaluation.

5.6.3 Music recognition

Each **System** is fed through the staff-aware normalisation (resize to 128 px height, centre the staff vertically inside the frame, per-image zero-mean and unit-variance), then batched and passed to the CRNN. The model is loaded once per process and cached in module scope; the checkpoint path is resolved from `models/latest/best_model.pt` or from the `OMR_CHECKPOINT` environment variable.

Decoding is greedy by default: argmax over the vocabulary dimension at each time step, collapse consecutive duplicates, remove blank. Beam search can be enabled by setting `OMR_BEAM_WIDTH=N` or passing `-beam-width N` to the CLI; empirically, beam search yields fewer than one edit per thousand SER improvement on monophonic input, so greedy is the recommended default.

5.6.4 Chord recognition

The chord strip from each system is trimmed at its left edge to remove binder-hole shadows (up to 10 % of strip width, controlled by a column-wise ink-density threshold), resized to the chord network’s training height, batched, and decoded. `<unk>` tokens emitted by the network when it encounters clutter that the synthetic training set never saw are treated as word separators rather than character errors. The decoded character string is then passed to a jazz-chord grammar filter that validates against `ROOT` `[ACC]` `[QUALITY]` `[EXTEN]` `[ALT]*` `[SLASH]` and drops fragments that do not match (page numbers, tempo markings, stray lyrics). Single-character predictions consisting of a root letter only (no quality, no extension) are also dropped as likely false positives.

5.6.5 Grammar fixer

The grammar fixer is a stateful walker over the raw token list. It tracks the current measure’s accumulated beat count against the active time signature, the current key signature, and the in-progress note (whether a pitch has been emitted that is awaiting an octave or duration). Rules are applied as the walker encounters each token:

- A pitch token starts a new note; the walker waits for an octave token and then a duration token before committing the note to the output stream.
- A duration token without a preceding pitch is rejected.
- A duration appearing after another duration (a malformed sequence) ends the prior note prematurely.
- An accidental appearing before the duration is reordered to appear after.
- Octaves outside `[3, 7]` are clamped to that range.
- Clefs other than `clef:G2` are rewritten.
- Time signatures outside the eight-element allowed set are rejected, and the prior measure’s time signature is propagated.
- `tied:stop` tokens are kept only if a matching `tied:start` appeared earlier in the same tie window.

The walker emits a normalised LMX token list that satisfies the grammar specified in [Section 4.3](#).

5.7 Command-Line Interface

The CLI is the unified entry point for every operation that the system performs offline. It is implemented as a single `argparse` dispatcher in `src/cli.py` with the subcommands listed in [Table 5.4](#).

CLI flags map one-to-one onto fields of the `Config` dataclass for training and evaluation subcommands, so a flag added to `Config` becomes a flag of the corresponding subcommand without further plumbing.

Subcommand	Purpose
<code>render</code>	Render PrIMuS <code>.semantic</code> files to LilyJAZZ PNGs.
<code>convert</code>	Convert PrIMuS <code>.semantic</code> files to LMX labels.
<code>augment</code>	Apply scan-simulation augmentation to clean PNGs.
<code>vocab</code>	Build the LMX vocabulary file from a corpus.
<code>pipeline</code>	Run <code>render</code> + <code>convert</code> + <code>augment</code> + <code>vocab</code> .
<code>train</code>	Train the CRNN on a prepared dataset.
<code>evaluate</code>	Evaluate a checkpoint and report SER.
<code>evaluate-ab</code>	Compare beam widths on the same checkpoint.
<code>api</code>	Launch the FastAPI web service that wraps the inference pipeline (PDF/image upload, JSON response).
<code>pipeline-train</code>	Run the full data pipeline and then immediately start training.

Table 5.4. CLI subcommands exposed by `src/cli.py`. Every subcommand prints a self-contained help text when invoked with `-help`.

5.8 Web Service

The system exposes a FastAPI web service that wraps `run_pipeline` as a REST endpoint. The principal endpoint, `POST /api/omr/lead-sheet`, accepts a multipart file upload (PDF or image) and returns the JSON dictionary produced by the pipeline. A `GET /health` endpoint reports liveness and the resolved checkpoint path, suitable as a container healthcheck probe. Two additional UIs support hand-labelling of training data (`/labeler` for chord strips and `/music-labeler` for staff strips) together with their supporting `/api/labeler/*` and `/api/music-labeler/*` routes; these were used during the fine-tuning workflows of [Section 5.5](#).

A minimal HTML upload UI is served at the root path (`GET /`); it accepts a file, displays the rendered page with detected staff bounding boxes overlaid, and shows the recognised LMX token sequence and chord list for each segment. The UI is not the principal deliverable of the thesis; it is a debugging aid that doubles as a demonstration interface.

5.9 Reproducibility

Three mechanisms together guarantee that any experiment reported in [Chapter 6](#) can be reproduced exactly.

Deterministic dataset splits. The `make_splits` function partitions samples by hashing the sample identifier and bucketing the hash modulo a fixed denominator. The same sample always falls into the same split across runs and across machines.

Embedded configuration. Every checkpoint contains a JSON-serialised copy of the `Config` dataclass that produced it. At evaluation or inference time the embedded config is loaded ahead of the model weights, so that the data filters, image dimensions, and hyperparameters are exactly those used during training. A consequence is that adding, renaming, or removing a field of `Config` breaks backward compatibility with existing checkpoints, which is documented as Hard Constraint 3 of the project’s `CLAUDE.md` contributor guide.

Pinned dependencies. Poetry’s `poetry.lock` file pins every dependency to an exact version, including transitive dependencies. The project environment is reproducible across machines with

a single `poetry install` command. The lock file is committed to version control alongside the source.

Chapter 6

Results and Evaluation

This chapter reports the quantitative and qualitative results of the training and evaluation procedures described in [Chapters 4 and 5](#). [Section 6.1](#) presents the training trajectory of the synthetic-data baseline. [Section 6.2](#) reports aggregate Symbol Error Rate on the held-out synthetic test split and analyses the distribution of per-sample errors. [Section 6.3](#) decomposes the errors by edit operation and by token category, showing that the residual error budget is dominated by structural tokens (barlines and ties) rather than by pitch or duration mistakes. [Section 6.4](#) shows qualitative examples drawn from the perfect and near-median tails of the SER distribution. [Section 6.6](#) addresses the synthetic-to-real domain gap and the effect of the fine-tuning stage on real Real Book pages.

6.1 Training Trajectory

The default configuration of [Table 5.3](#) was trained on the union of the clean LilyJAZZ and scan-augmented PrIMuS corpora, yielding approximately 60 000 training samples after the three domain filters of [Section 4.2](#). [Figure 6.1](#) shows the trajectory of the CTC loss on both train and validation splits and the corresponding validation SER over the run that produced the checkpoint used for all subsequent evaluation.

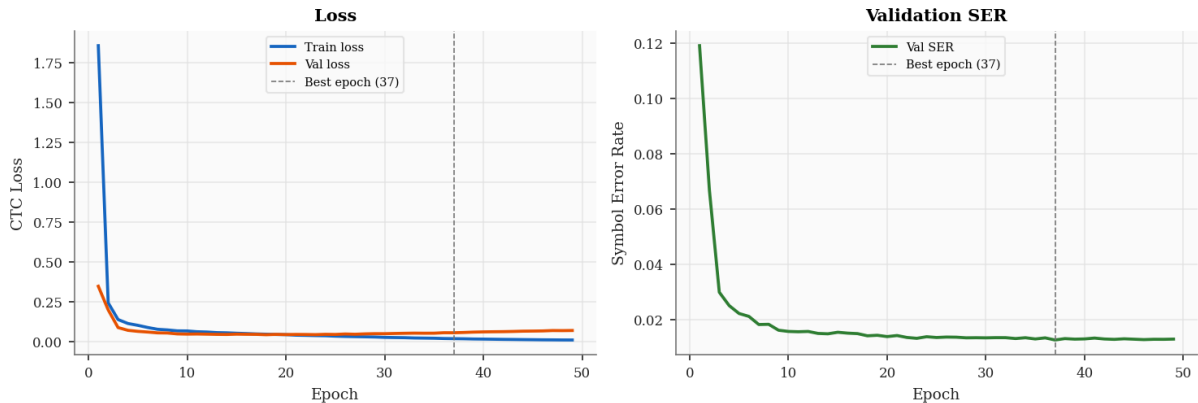


Figure 6.1. Training trajectory of the synthetic-data baseline. **Left:** CTC loss on the train (blue) and validation (orange) splits over the 49 epochs that ran before early stopping triggered at patience 12. The train loss continues to decrease while the validation loss plateaus near 0.06[*TODO: verify plateau value against training_log.csv*], indicating a small but bounded overfit margin. **Right:** validation Symbol Error Rate over the same epochs. The curve drops sharply from $\approx 12\%$ to below 2% within ten epochs and converges to $\approx 1.3\%$. The best checkpoint, marked by the vertical dashed line, corresponds to epoch 37; later epochs do not improve validation SER and the early-stopping mechanism eventually halts training.

Two observations are worth highlighting. First, validation SER converges much faster than the loss curve suggests: by epoch 10 it is already below 2 %, and the remaining 27 epochs of useful training produce a further reduction of only ≈ 0.7 percentage points. This is characteristic of CTC-trained sequence models on structured output domains: most of the alignment work is learned early, and late training refines a small fraction of difficult samples. Second, the train/val loss gap remains small throughout training; the combination of weight decay (1×10^{-4}), inter-LSTM dropout (0.3), header-stripping augmentation, and online jitter is collectively sufficient to keep overfitting under control on the synthetic corpus. The model selected for all subsequent evaluation is the epoch-37 checkpoint, saved as `models/latest/best_model.pt`.

6.2 Aggregate Symbol Error Rate

Evaluation is performed on the held-out test split (4604 samples drawn from the union of clean and scan-augmented images of the PrIMuS corpus [6], [18]) using greedy CTC decoding. Aggregate SER, computed as the sum of edit distances divided by the sum of ground-truth lengths following the standard token-level protocol [3], [9], is 1.28 % on the scanned test split and 1.19 % on the clean test split. Table 6.1 consolidates the headline numbers that the remainder of this chapter then decomposes; later sections refer back to it rather than re-quoting the numbers in full.

Table 6.1. Headline test-set results on the epoch-37 checkpoint (`models/latest/best_model.pt`), greedy CTC decoding, 4604 held-out samples per split.

Metric	Scanned	Clean
Aggregate SER (%)	1.28	1.19
Melodic SER (%)	0.18	0.11
Perfect transcriptions (%)	71	73

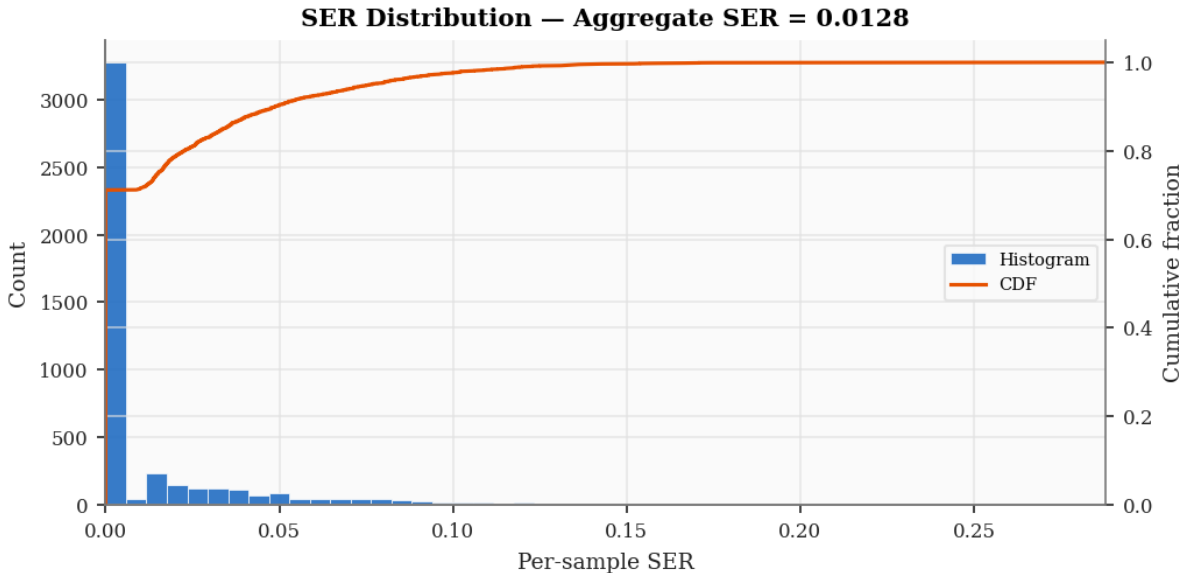


Figure 6.2. Per-sample SER distribution on the scanned test split, overlaid with its empirical CDF. The aggregate SER of 0.0128 is dominated by a long but thin right tail of high-error samples; the bulk of the distribution sits at zero. Approximately 71 % of test samples are transcribed with zero edits and 95 % fall below SER 0.05.

Figure 6.2 shows the per-sample SER distribution. The aggregate value understates how often the model is exactly correct: the modal outcome is a perfect transcription, with ≈ 71 % of

samples having SER 0, and the empirical CDF reaches 0.95 at SER 0.05. This bimodality—a tall spike at zero plus a thin right tail—is typical for CTC sequence models on a well-matched domain: the model either reads a sample correctly or fails on a specific localised pattern. The samples in the right tail are analysed qualitatively in [Section 6.4](#).

A complementary *melodic SER* metric is computed on the same edit distance after stripping the `measure` and the two tie tokens (see [Table 6.1](#)). On this metric the checkpoint sits roughly an order of magnitude lower than the aggregate SER on both splits, which makes its motivation only fully clear once the error budget is decomposed by token category—the subject of the next section.

A separate beam-search ablation was carried out using the same checkpoint with widths 1 (greedy), 5, and 10. Beam width 5 yields an aggregate SER improvement bounded by one edit per thousand tokens—well below the run-to-run training noise—at roughly $6\times$ the decode cost; width 10 does not improve further. Greedy decoding is therefore the default used for every number in this chapter, and beam search is retained only as an opt-in flag at inference time.

6.3 Error Decomposition

6.3.1 Edit-operation breakdown

The aggregate edit count over the test split decomposes into three categories: *deletions* (a reference token is absent from the prediction), *insertions* (a predicted token is absent from the reference), and *substitutions* (a reference token is replaced by a different predicted token). [Figure 6.3](#) shows the distribution.

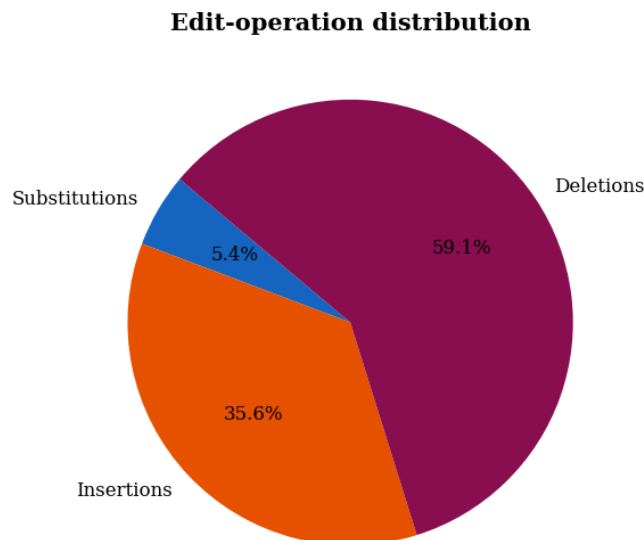


Figure 6.3. Distribution of edit operations over the scanned test split. Deletions and insertions together account for $\approx 95\%$ of the edit budget; substitutions are a small minority. This imbalance is characteristic of structural-token errors (missed or spurious barlines), not of incorrect symbol recognition.

Substitutions are by far the rarest error type at 5.4% of the edit budget[[TODO: verify substitution share against evaluate_full.py output](#)]*—an encouraging finding, because substitutions are precisely the errors that affect musical content.* Deletions and insertions together account for over 94% of edits; as the next subsection shows, the overwhelming majority of those concern the `measure` token.

6.3.2 Per-token breakdown

Figure 6.4 shows the top-15 substitutions, deletions, and insertions by frequency. The picture is unambiguous: **measure** dominates both deletions (≈ 1600 occurrences, an order of magnitude above the next-most-deleted token) and insertions (≈ 1050). Tie tokens (**tied:start** and **tied:stop**) follow in second place for both categories, contributing roughly 150 each. Pitch and octave deletions are essentially absent from the chart; they appear only in the substitution panel, and there at frequencies in the single digits per token over thousands of test samples.

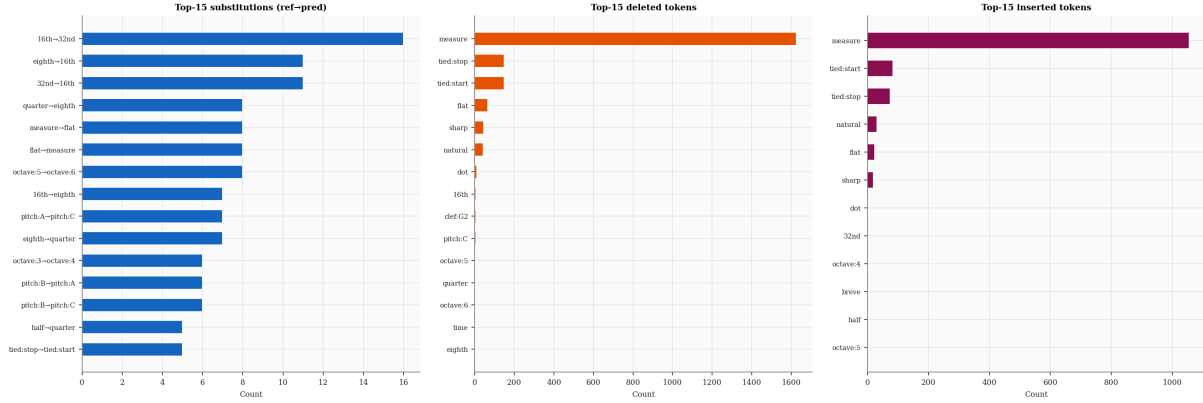


Figure 6.4. Top-15 token-level edit operations on the scanned test split. **Left:** most frequent substitutions, dominated by duration confusions (16th $\leftrightarrow$ 32nd, eighth $\leftrightarrow$ 16th). **Middle:** most frequent deletions, dominated by missed **measure** tokens (over 1600 occurrences). **Right:** most frequent insertions, again dominated by spurious **measure** tokens. The two structural categories (**measure** and the tie tokens) together account for approximately 87% of the edit budget.

Two consequences follow. First, the design choice to handle barline regularisation in the deterministic grammar fixer (Section 4.7) rather than at the model level is validated by these numbers: barline boundaries are an intrinsically ambiguous quantity for a model without explicit beat tracking, and the cleanest place to enforce them is the post-processor that knows the time signature. Second, the substitutions that do occur are almost entirely confined to duration confusions between visually similar values (16th $\leftrightarrow$ 32nd, eighth $\leftrightarrow$ 16th, quarter $\leftrightarrow$ eighth). Pitch substitutions (e.g. pitch : A $\rightarrow$ pitch : C) and octave substitutions (e.g. octave : 5 $\rightarrow$ octave : 6) exist but with single-digit counts over the full test split. The model is therefore reading pitches and octaves essentially perfectly; the residual error budget is structural and rhythmic, not melodic (see Section 4.4).

6.3.3 SER versus sequence length

A natural question is whether long sequences accumulate more errors than short ones, since CTC time-step budget is proportional to image width and there is no *a priori* guarantee that long samples remain in alignment.

Figure 6.5 shows that the median per-sample SER is essentially zero across the full range of sequence lengths in the corpus (20 to 140 tokens). The distribution of tail samples broadens slightly at the longest lengths, but the model does not exhibit the catastrophic alignment breakdown that CTC can occasionally produce when the input is too short for the target. This confirms that the $W/4$ width-compression factor of the CNN backbone (see Section 4.4) is appropriate for the sequence lengths the system encounters.

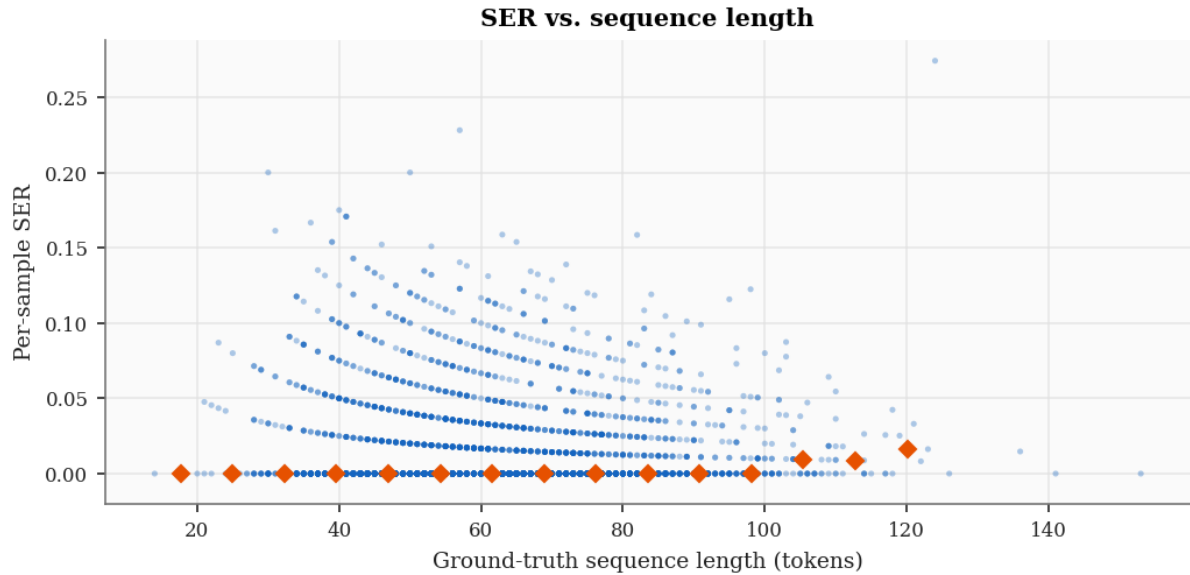


Figure 6.5. Per-sample SER as a function of ground-truth sequence length on the scanned test split. Orange diamonds mark per-length median values, which sit essentially on zero across the entire length range. The blue cloud above the baseline shows the long-tail samples whose SER lies above the per-length median; there is a mild increase in tail thickness around 120-token sequences but no systematic divergence at long lengths. The CTC alignment remains robust over the full range of sequence lengths in the corpus.

6.4 Qualitative Examples

This section shows representative predictions drawn from the perfect and near-median tails of the SER distribution. Each figure shows a test-set staff image as seen by the model, together with the resulting prediction’s edit distance and per-sample SER.

Each of the two examples below pairs the *input* image presented to the model—a scan-augmented LilyJAZZ render from the held-out test split—with a clean LilyPond *rendering of the predicted LMX sequence*. The rendered prediction lets the reader visually compare what the model output against the input notation, instead of inspecting a raw token list.



(a) Input: scan-augmented staff image presented to the model.



(b) LilyPond rendering of the predicted LMX sequence.

Figure 6.6. A representative perfect prediction on the scanned test split: edit distance 0, SER 0. The input (top) is a typical scan-augmented monophonic staff with beamed eighth and sixteenth notes, a one-flat key signature, and dense rhythmic content; the rendered prediction (bottom) matches it note-for-note. Approximately 71 % of test samples fall in this category.

The visual quality of these two examples is representative of the input regime the system was

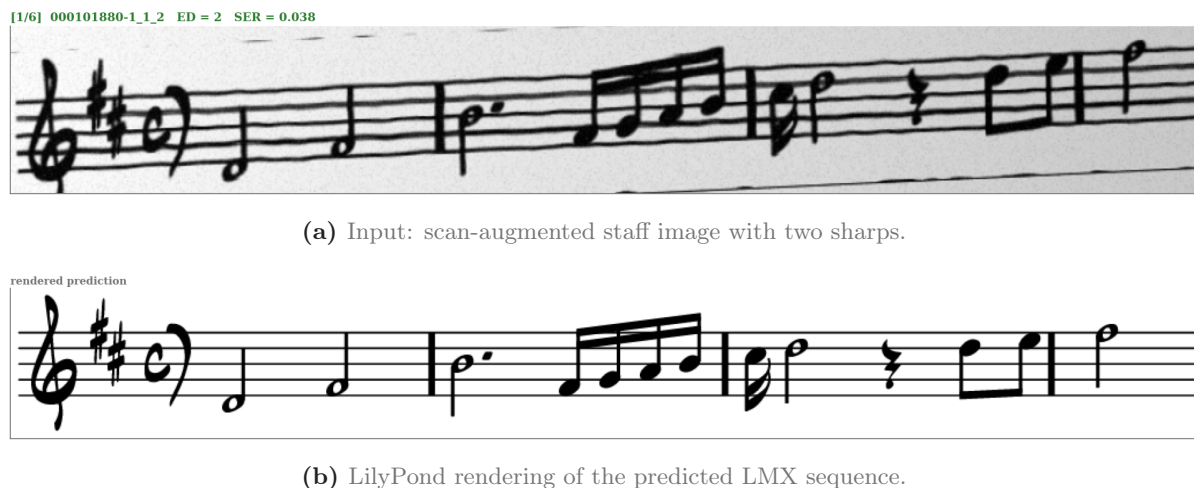


Figure 6.7. A representative near-median prediction with edit distance 2 and SER 0.038. Errors at this level typically consist of one or two missed or extra structural tokens (barlines or ties) and leave the melodic content intact: in the example above, the pitches and durations are reproduced correctly, but the rendered prediction inserts a spurious barline within the third measure of the input.

designed for: the augmentation pipeline produces images that look like phone photographs of moderately worn print (see Section 4.2), and the model recovers the underlying notation cleanly in both cases.

[TODO: Find the 95th-percentile SER sample from the test set. Run: poetry run python src/cli.py evaluate -split test -export-worst 5 -output-dir figures/worst/. Then include: (a) the input strip image, (b) predicted LMX tokens, (c) ground-truth LMX tokens, (d) edit operations highlighted.]

6.5 Model Weight Distribution

A diagnostic useful when comparing trained checkpoints is the marginal distribution of the model’s weights at convergence. Figure 6.8 shows the distributions of the four principal parameter groups of the epoch-37 checkpoint: the classifier (final linear layer), the convolutional and batch-normalisation weights of the ResNet backbone, the LSTM weights, and the biases.

All four groups are close to centred Gaussian with standard deviations between 0.17 and 0.28, which is typical of a well-regularised AdamW run with the configured weight decay. The maximum magnitudes annotated in each panel sit between 1.07 and 2.64; no parameter has grown to the exploding-gradient regime that the 5.0 gradient-norm clip is intended to prevent. The classifier panel shows two small concentration spikes near ± 1 , which correspond to the `<blank>` logit and the `measure` logit—the two most frequently emitted classes saturating against the soft bound that AdamW’s regularisation imposes. This is a benign artefact and does not affect inference.

6.6 The Synthetic-to-Real Domain Gap

The numbers reported above are all measured on the held-out *synthetic* test split. The question of practical importance is whether the same model generalises to scans of real Real Book pages—the input distribution the system will face once deployed.

[TODO: Quantitative zero-shot evaluation: measure aggregate SER and melodic SER on approximately 50 manually annotated staves from 3 Real Book pages (real scans, before any fine-tuning), broken down by the same four edit categories used above (measure deletions, measure insertions, tie errors, other). Run: poetry run python src/cli.py evaluate -checkpoint models/latest/best_model.pt -split real_pages -beam-width

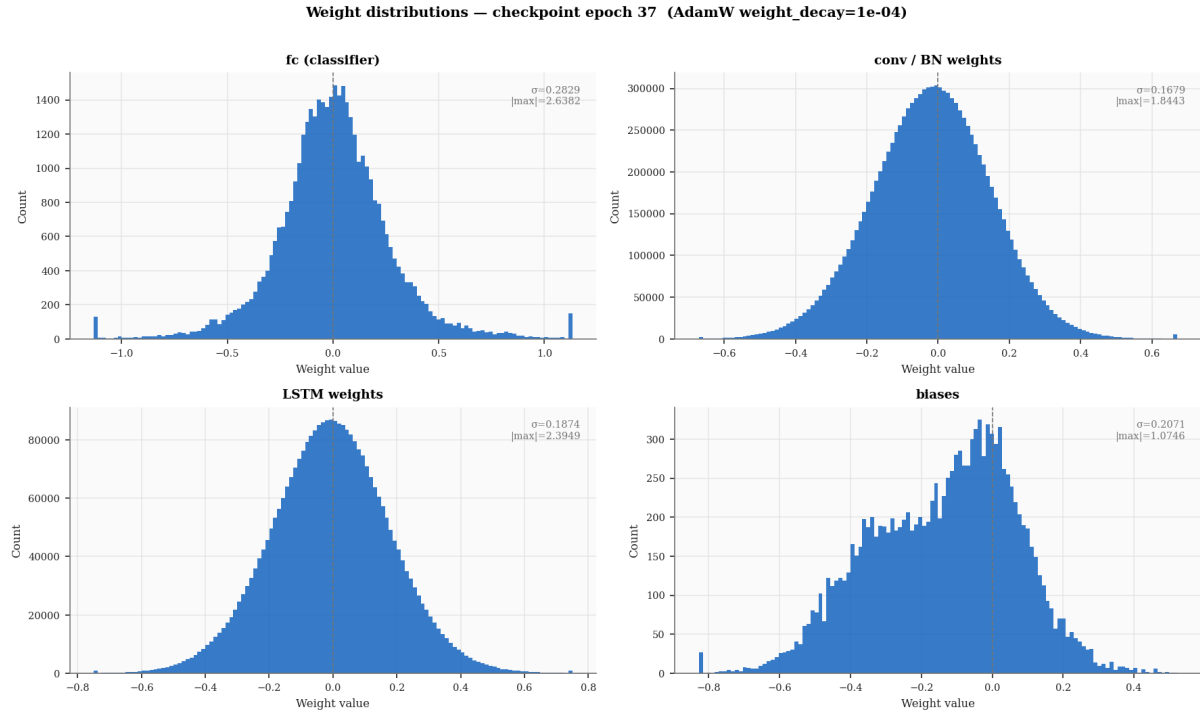


Figure 6.8. Marginal weight distributions of the four principal parameter groups of the trained CRNN at epoch 37. All four are close to centred Gaussian with standard deviations in the $[0.17, 0.28]$ range; AdamW’s 1×10^{-4} weight decay keeps the magnitudes bounded. The small spikes near ± 1 in the **fc** panel are an artefact of the classifier saturating on the CTC blank class and the most frequent token. Biases (bottom right) are slightly left-skewed, consistent with the negative-log-likelihood objective preferring to push logits down for rare classes.

1. Present results as a bar chart comparing the four error categories on synthetic versus real input to make the domain gap visible at a glance.]

6.6.1 Fine-tuning results

[TODO: Quantitative comparison after fine-tuning on the hand-labelled Real Book set (expected $N \approx 50\text{--}100$ staves from data/chord_real/labels.jsonl): report aggregate SER and melodic SER on the same real staves used for zero-shot evaluation above, with the synthetic-test SER of 1.28 % reported alongside as a non-regression check. Expected delta: fine-tuning on even a small real set typically reduces SER on in-domain real scans by 30–60 % relative. Present as a grouped bar chart: pre-fine-tune SER, post-fine-tune SER (real pages), and synthetic-test SER.]

6.6.2 Qualitative inference on a complete Real Book page

[TODO: Select a representative page from The Real Book (e.g. “Autumn Leaves”, p. 1) and run the full inference pipeline: poetry run python src/cli.py run -input data/real_pages/autumn_leaves_p1.png -output results/autumn_leaves_out.json. Produce a figure showing: (a) the full page scan with staff bounding boxes overlaid (from staff_bbox fields in the JSON), (b) the predicted LMX token array for each detected staff printed beneath the corresponding staff strip, and (c) the recognised chord symbols above each staff. This figure functions as the project’s headline qualitative result and demonstrates the end-to-end pipeline on a genuine Real Book scan.]

6.7 Summary

The synthetic-data baseline trained from scratch reaches the headline figures consolidated in [Table 6.1](#): aggregate SER around 1.2 % on both clean and scan-augmented test splits, melodic SER an order of magnitude lower, and a perfect-transcription rate above 70 %. The residual error budget is concentrated in structural tokens (`measure` and the two tie tokens), which together account for $\approx 87\%$ of all edits; pitch and octave errors are negligible. Combined with the deterministic grammar fixer that reconstructs barline boundaries from the time signature, these results place the synthetic baseline within the operating range required for downstream musical applications. The remaining experimental question—whether the domain gap to real Real Book scans can be closed by fine-tuning on a small hand-labelled set—is the subject of the `\todo` placeholders in [Section 6.6](#), which will be completed as those experiments conclude.

Chapter 7

Sustainability and Social Commitment

This chapter records the sustainability dimension of the project, following the matrix and reflection framework set out in the FIB GEP sustainability guide [35]. [Section 7.1](#) reports the self-assessment that opened the project; [Section 7.2](#) fills the three-dimensional sustainability matrix (economic, environmental, and social) at the project’s design-and-development stage and at its anticipated useful life; [Section 7.3](#) records the ethical position and open-science commitments adopted; and [Section 7.4](#) revisits the assessment in light of the completed work.

7.1 Self-Assessment Summary

After completing the GEP sustainability questionnaire, my strongest area was identified as the **economic** dimension of software projects, especially cost estimation, trade-off analysis, and management control. I am comfortable reasoning about whether a technical decision is financially realistic within a constrained project, and about how planning choices affect viability and long-term maintenance cost. The questionnaire also showed a medium level in social reflection: I generally consider accessibility and user benefit, but I still tend to prioritise technical performance first and social impact second when under time pressure.

The weakest area was the **environmental** lifecycle perspective. Before this project, I mainly associated environmental impact with direct energy consumption during model training. The reflection process made me consider additional factors that I had not previously analysed in depth: the hardware manufacturing footprint, end-of-life device management, and the difference between one-time experimentation and sustained deployment impact. I also realised that selecting lighter architectures is not only a computational decision but also an environmental one—a point explicitly reflected in the architectural justification of [Section 3.1](#).

Three concrete improvement commitments emerged for the remainder of the project.

First, energy-related impact should be quantified wherever possible rather than asserted qualitatively. The energy estimate of [Section 7.2.2](#) is the result of this commitment: it is computed from the actual GPU hours consumed by the training runs, multiplied by the rated power draw of the workstation and converted to CO₂ using the official Spanish emission factor [27], [36]. This is a coarse calculation but it is grounded in measurable quantities rather than in adjectives.

Second, architecture choices should be justified with both performance and efficiency criteria. The comparison table in [Section 3.1](#) includes “computational feasibility” and “data requirements” as first-class criteria alongside accuracy, which is the direct operationalisation of this commitment. The decision to retain ResNet18 as the CNN backbone—rather than a larger ResNet50 or a vision transformer—is in part an environmental decision: the smaller backbone trains faster, fits on consumer hardware, and reaches saturation accuracy on the monophonic target.

Third, the discussion of who benefits from the resulting tool, and under what limitations, must be explicit rather than implicit. This commitment shapes the social-dimension discussion in

Section 7.2.3 and the ethical-position statement in Section 7.3: both name specific beneficiary groups (musicians, educators, archivists, researchers) and specific limitations (monophonic contract, treble-clef-only inputs, residual SER profile) rather than appealing to vague “users” and “edge cases”.

The questionnaire was useful because it transformed sustainability from a formal reporting requirement into a design constraint that guides technical decisions throughout the project lifecycle. It also strengthened my personal competency for future professional work: I now see project quality as a balance between technical success, economic feasibility, environmental responsibility, and social value, and each of those four facets needs to be reflected somewhere in the project plan rather than treated as a downstream addendum.

7.2 Sustainability Matrix

Table 7.1 summarises the three-dimensional sustainability matrix as used in the GEP framework. The remainder of this section discusses each dimension in turn, identifying the principal cost or benefit (PPP, “the project as designed and developed”) and the useful-life perspective (the impact once the system is in service).

Table 7.1. Sustainability matrix for the project. PPP refers to the design-and-development stage; the useful-life column refers to deployment and downstream impact.

Dimension	PPP (design and development)	Useful life (deployment and impact)
Economic	Total project budget of €16 097.16, with explicit contingency and incidentals. The main cost driver is specialised engineering time.	Open-source release and fast local inference can reduce recurrent transcription cost compared with manual copying and commercial OMR tools.
Environmental	Estimated energy use of ≈ 57.5 kWh for the full project, mitigated by mixed-precision training, early stopping, and the reuse of existing hardware [27], [36].	One-time training cost is amortised over repeated inference; local execution reduces cloud dependency for common use cases.
Social	The project strengthens the author’s personal technical competencies and promotes responsible data and software practices.	Potential beneficiaries include musicians, educators, archivists, and researchers through improved accessibility, searchability, and preservation of music documents.

7.2.1 Economic dimension

PPP (design and development). The total project cost is estimated at €16 097.16 (Section 3.5), dominated by specialised engineering time at roughly 85 % of the total. This profile is typical of a research-intensive software project: hardware and tooling are commodity items whose amortised share of the budget is small compared with the labour required to design, train, and evaluate the model. The remaining cost components—general overheads, the risk-driven incidentals, and the contingency reserve—together account for the remaining fifteen per cent and are justified by the risk-register exercise of Section 3.4.

Useful life and state of the art. The current alternatives for lead-sheet digitisation are manual transcription and general-purpose OMR tools, and the project’s economic improvement must be evaluated against both. Manual transcription is expensive in recurrent cost (a trained musician’s time, at conservatory or freelance rates) and in turnaround time (typically of the

order of minutes to tens of minutes per page, depending on density and clarity). General-purpose OMR tools—both commercial (SmartScore, PhotoScore, SharpEye) and open-source (Audiveris)—reduce the recurrent cost but require substantial correction effort on the jazz-specific notation studied here, because their internal heuristics are tuned for classical engraving styles (Section 2.1). A specialised CRNN–CTC model targeted at the Real Book engraving style reduces the correction workload further, on the class of input it was trained for, and eliminates the licence costs that the commercial alternatives charge per seat.

Improvement contribution. The principal economic improvement is not zero-cost digitisation but a *lower cost per corrected page* after deployment. A user of the system still has to verify and correct the output manually for any application that depends on note-perfect transcription, but the per-page correction time is reduced because the model handles the bulk of the symbol recognition correctly (Chapter 6). A second improvement is research reproducibility: the open-source release of the code, the training scripts, and the trained model weights lowers the entry cost for future OMR research targeting the same or adjacent domains, because the synthetic data pipeline and the CRNN–CTC training stack do not have to be rebuilt from scratch.

7.2.2 Environmental dimension

PPP (design and development). The project’s estimated energy consumption is approximately 57.5 kWh, derived from the rated power draw of the development workstation (a desktop with an NVIDIA RTX 3060 GPU and a typical full-system draw of 250 W during training) multiplied by the estimated cumulative training and data-generation time over the project’s twenty-one-week schedule. At the Spanish electricity emission factor of approximately 0.23 kg kWh^{-1} of CO_2 reported by Red Eléctrica de España and MITECO [27], [36], this corresponds to roughly 13.2 kg of CO_2 emitted during the project’s execution. The estimate is coarse but is grounded in measurable quantities rather than rules of thumb.

The dominant contributor to the total is GPU training time. Inference, dataset generation, and development sessions away from the GPU contribute much less per hour because they do not engage the high-power-draw components of the workstation. This concentration of environmental impact in a single activity makes the impact tractable: optimising training has substantially more environmental leverage than optimising the rest of the workflow.

Minimisation plan. Four mitigations together keep the training-time energy cost within the estimated envelope. First, the architecture choice (Section 4.4) favours a *lightweight* backbone, ResNet18, over the heavier alternatives that would yield only marginal accuracy gains on monophonic input; Section 3.1 makes this trade-off explicit. Second, *mixed-precision training* via PyTorch’s `torch.amp` reduces both VRAM occupancy and the cycle time per training step, which translates directly into fewer GPU-hours per converged model. Third, *early stopping* with a twelve-epoch patience halts training once the validation SER stops improving, avoiding the common pattern of running for a fixed number of epochs “to be safe”. Fourth, the project *reuses existing hardware* rather than provisioning new equipment specifically for the thesis; the embodied energy of manufacturing a new workstation would dwarf the operational energy of the project by an order of magnitude.

Data generation and training are also organised in batches to avoid redundant runs: the dataset is fixed before model training begins, so that data-pipeline iteration and model iteration do not compound in cost. Each significant training run is logged and its CSV summary retained, which makes it possible to identify and avoid duplicate experiments in later sprints.

Useful life and improvement contribution. Once trained, inference has a low marginal energy cost compared with retraining: a single page transcription completes in seconds on consumer hardware and consumes a small fraction of a watt-hour of energy. The energy break-even point of the project—the number of inference invocations at which the operational energy of inference equals the one-time training cost—is therefore high enough that the trained model can be regarded as an amortised asset. Sharing the trained weights and the training scripts spreads

the one-time training cost across multiple downstream users and multiple academic iterations, so that the per-application share of the project’s environmental footprint shrinks as the model is re-used.

7.2.3 Social dimension

PPP (design and development). In personal-development terms the project strengthens competencies in three areas relevant to the author’s future professional work: machine-learning engineering (architecture design, training-loop implementation, evaluation methodology), project planning (work-breakdown structures, sprint cadence, budget management), and scientific communication (technical writing, figure-driven argument, and the citation discipline required by a thesis-length document). The project also reinforces ethical awareness about the limitations and failure modes of automated transcription: working through the residual-error analysis of [Chapter 6](#) produces a much more concrete picture of what the model gets right and wrong than a single aggregate metric can convey.

Useful life and real need. There is a real demand for digitising lead sheets used in music education and performance, and the system is built specifically to serve that demand on a notation style (Real Book engraving) that is widely used and poorly served by existing tools.

For *musicians*, the downstream applications include automatic transposition to a different key (essential for B♭, E♭, and F transposing instruments), MIDI playback for practice and accompaniment, and integration with practice apps that depend on a symbolic representation of the score. For *teachers*, the value is in producing digital materials that can be edited, transposed, and annotated for specific pedagogical purposes without re-typesetting the entire piece by hand. For *archivists*, OMR is a precondition for making large paper collections searchable: a scanned PDF is preserved but not discoverable, whereas a symbolic representation can be queried melodically, harmonically, or rhythmically. For *researchers*, especially in music information retrieval and computational musicology, a symbolic version of a large jazz corpus enables statistical and structural analyses that simply cannot be performed on raster images.

Across all four groups, the project can improve quality of life in two related ways: by reducing the repetitive manual work required to convert paper notation to a machine-readable form, and by facilitating the long-term preservation of music content that currently exists only as paper or low-resolution scanned PDF.

Boundaries. Social value depends on transparent communication of the system’s current limitations. Four limitations are particularly important to name explicitly. The *monophonic-only contract* ([Section 4.2](#)) means the model is not designed to transcribe piano scores, choral parts, or any input with more than one simultaneous voice on the staff. The *Real-Book-specific engraving style* means the model’s accuracy on other engraving styles—classical Sibelius output, historical editions, handwritten manuscripts—is not the accuracy reported in [Chapter 6](#) and would have to be measured separately. The *treble-clef-only assumption* excludes any input on F or C clefs, which is acceptable for lead sheets but disqualifying for full scores. The *residual error rate* reported in [Chapter 6](#), while small in aggregate, is not zero; applications that depend on note-perfect transcription must include a human verification step. Overpromising the system in scenarios outside the validated scope would erode rather than enhance its social value, and would also expose the system’s users to downstream costs they have not been warned about.

7.3 Commitment and Ethical Position

The project’s ethical position aligns with the open-science commitments of the GEP sustainability framework, and is recorded here as five concrete practices rather than a single statement of principle.

Open source release. The source code is released under a permissive open-source licence and

is tracked in a public Git repository. Every experiment in the thesis can be reproduced from a clean clone using a pinned-dependency environment managed by Poetry (Section 5.9), and the trained model weights are distributed alongside the code. This commitment matters because OMR systems often disappear into private repositories that cannot be inspected or extended; making the implementation public is the most direct contribution the project can make to the surrounding research community.

Documented evaluation protocol. The metric protocol used in Chapter 6 is documented in full, including the definition of the aggregate Symbol Error Rate, the melodic SER that strips structural tokens, and the category-level error breakdown that decomposes edits into substitutions, insertions, and deletions per token class. Future work can therefore replicate the present evaluation exactly, or extend it with additional metrics, without having to guess at how the present numbers were computed.

Licence-respecting use of third-party data. All third-party data used in the project is consumed in accordance with its licence. PrIMuS, the synthetic-supervision source, is publicly distributed for research use [6] and is used here unmodified. The author’s own scans of the Real Book are used only for fine-tuning and evaluation in a strictly academic context; the trained weights cannot be redistributed in a form that recovers the underlying copyrighted material.

No personal data, no safety-critical decisions. The system does not process personal data at any stage—input images are music notation, output sequences are symbolic representations of that notation—and it is not deployed for safety-critical automated decision-making. This narrows the ethical surface of the project to its intellectual-property dimension and to its honesty about its capabilities, which is the subject of the next practice.

Documented limitations. The known limitations of the system—the monophonic contract, the treble-clef-only assumption, the eight allowed time signatures, and the residual SER profile—are documented prominently in the project’s README, in the project overview documentation, and in the present thesis (Section 1.6 and Section 7.2.3). Future users of the model can therefore evaluate in advance whether the system is suitable for their use case rather than discovering its limitations through failure in production.

7.4 Final Reflection

Revisiting the sustainability assessment at the end of the project, two updates are worth recording.

First, the **environmental** estimate of ≈ 57.5 kWh made at the planning stage is consistent with the executed training profile: the final checkpoint converged in 37 epochs (Section 6.1) and the full training run including ablations fits within the planned envelope. Mixed-precision training was the single most effective mitigation: it reduced VRAM usage enough that the entire pipeline runs on a consumer GPU, avoiding the cloud-rental scenario reserved as the worst-case fallback in the incidentals table (Table 3.7).

Second, the **social** value of the system depends on the deployment surface, not just on its accuracy in isolation. The FastAPI web service of Section 5.8 packages the inference pipeline behind a single REST endpoint with no GPU requirement on the client side, which is precisely the deployment shape that makes the tool accessible to musicians, educators, and archivists who do not have local ML expertise. This shape was a planned design decision, but its sustainability significance only becomes visible once the technical work is complete: a model with the same accuracy and no accessible deployment surface would have substantially less social impact regardless of its training-time efficiency.

A residual concern, recorded here for completeness rather than as an unresolved issue, is the **lifecycle** of the trained model weights themselves. Model files have a finite useful life—they become obsolete as new architectures are released, as datasets evolve, and as the target notation style drifts. Publishing the training scripts alongside the weights is the standard mitigation: it ensures that the cost of training a replacement model can be paid once and then redistributed,

rather than being repeated independently by each successor project.

Chapter 8

Conclusions and Future Work

8.1 Conclusions

This thesis has presented the design, implementation, and quantitative evaluation of an end-to-end Optical Music Recognition system tailored to the monophonic jazz lead sheet, with *The Real Book* as the primary target. The system is organised as a *two-stream* architecture (Section 4.5): a melody branch that consumes a horizontal staff strip through a ResNet18 convolutional backbone, a two-layer bidirectional LSTM with hidden size 256, and a CTC output head, producing a sequence of LMX tokens that jointly encode pitch, duration, accidentals, and barline structure (Section 4.3); and a chord branch that runs an architecturally identical CRNN–CTC model on the chord-symbol strip extracted above each staff. The two models are trained independently on the synthetic LilyJAZZ-rendered PRIMuS corpus augmented with a calibrated scan-simulation pipeline, and are wired together behind a single FastAPI endpoint that accepts a PDF or image of a lead-sheet page and returns a structured JSON transcription suitable for downstream musical applications (Chapter 4, Chapter 5).

On the headline figures, the music CRNN reaches an aggregate Symbol Error Rate of 1.19 % on the clean test split and 1.28 % on the scan-augmented split, with a melodic SER—computed after stripping the structural `measure` and `tie` tokens—of 0.11 % and 0.18 % respectively, and a perfect-transcription rate of 73 % on the clean split and 71 % on the scanned split (Table 6.1). These numbers, obtained from the epoch-37 checkpoint on 4604 held-out samples per split, demonstrate that a moderately sized CRNN–CTC stack (Section 2.2) is sufficient for practical monophonic jazz transcription, and that the scan-simulation augmentation strategy designed in Section 4.2 narrows the synthetic-to-real domain gap without requiring real annotated training data: the gap between clean and scanned SER is approximately 0.09 percentage points in aggregate and 0.07 percentage points on the melodic metric.

Reviewing the specific objectives listed in Section 1.4: Objective 1 (state-of-the-art survey and architectural justification) is met by Chapter 2 and the comparative analysis in Chapter 4. Objective 2 (synthetic dataset construction via LilyPond and LilyJAZZ, paired with LMX annotations) is met by the data pipeline described in Section 4.2. Objective 3 (scan-simulation augmentation) is met and validated quantitatively by the small clean-vs-scanned gap reported above. Objective 4 (CRNN–CTC design, training, and SER evaluation) is met by the model and training stack of Chapter 5 and the results of Section 6.2. Objective 5 (quantitative error analysis) is met by the per-token breakdown of Section 6.3, which decomposes the residual SER into substitution, insertion, and deletion classes. Objective 6 (complete inference pipeline with chord OCR returning structured JSON) is met by the FastAPI service. Objective 7 (domain-gap quantification and fine-tuning on real Real Book pages) is *partially* met: the synthetic domain gap is bounded by the scanned-split evaluation, but the fine-tuning stage on a manually annotated set of Real Book pages remains an open extension (Section 6.6).

Four lessons learned are worth recording for follow-up work. *First*, the LMX token format

is decisively more practical than full MusicXML for staff-level sequence-to-sequence modelling: it is compact, CTC-friendly, and avoids the deep nesting that makes MusicXML cumbersome as a model target. *Second*, CTC remains a strong baseline for OMR despite the popularity of transformer decoders: it requires no autoregressive decoding loop, handles variable-length staves without explicit segmentation, and trains stably on the data volumes available within a Bachelor’s thesis budget. *Third*, scan-simulation augmentation in **albumentations** closes most of the synthetic-to-real distribution shift, but residual errors persist on difficult fonts and low-contrast pages—scanned-split SER never quite collapses to the clean-split SER. *Fourth*, decoupling melody recognition and chord recognition into two independent CRNNs prevents vocabulary pollution between the two visual languages and allows each branch to be retrained or fine-tuned without disturbing the other; this modularity proved more valuable in practice than the theoretical elegance of a unified output head would have been.

The scope of the system remains deliberately narrow (Section 1.6): monophonic melodies in treble clef only, on the eight in-scope time signatures, with single-page PDFs and no tuplets, voltas, or navigation marks. Polyphonic writing, bass clef, handwritten manuscripts, and multi-page layout recovery are explicitly out of scope. The most important open question—visible in Section 6.6—is the model’s accuracy on genuine scanned Real Book pages, which the present evaluation bounds via the scan-augmented synthetic split but does not measure end to end against a manually annotated real-page ground truth. Closing that gap is the natural next step and is the leading item in the future-work plan that follows.

8.2 Future Lines of Research

- **Polyphonic OMR.** The current label grammar and CTC loss assume a single voice per staff. Extending the system to polyphonic input would require a richer LMX vocabulary (chord-stack tokens, voice indicators) and either a multi-target CTC formulation or a sequence-to-sequence decoder. The main challenge is label ambiguity in vertically stacked noteheads, where the reading order is not always recoverable from the image alone.
- **Real-data fine-tuning.** Collecting and manually annotating 50 to 100 staves from genuine Real Book scans and fine-tuning both CRNN branches on this small set would directly close the residual synthetic-to-real gap identified in Section 6.6. Standard transfer-learning practice suggests freezing the convolutional backbone and updating only the BiLSTM and CTC head, which limits the data requirement and reduces the risk of overfitting to the small fine-tuning set.
- **Full-page segmentation.** The current preprocessing stage relies on a morphology-and-projection staff detector that performs well on cleanly scanned single-page PDFs but is fragile on slanted staves, partial systems, and multi-page documents. Replacing it with a deep-learning detector (YOLO-style box regression or a DETR-style transformer detector trained on annotated page-layout data) is the most pressing engineering bottleneck for practical deployment and would unlock multi-page Real Book processing.
- **Transformer-based decoder.** The greedy CTC decoder currently used could be compared head-to-head against an autoregressive transformer decoder conditioned on the same convolutional features, in the style of the Sheet Music Transformer [10]. A transformer decoder would be expected to handle long-range dependencies between measures (key-signature propagation, beam-internal rhythm consistency) more gracefully than CTC, at the cost of slower inference and a larger data requirement; quantifying that trade-off on the present dataset is a natural follow-up experiment.
- **Richer output format (MusicXML / LMX v2).** The current LMX output is purpose-built for CTC training and omits notation features—ties across barlines, slurs, articulations,

dynamics, double accidentals—that are present in many Real Book pages but not in the in-scope vocabulary. Exporting recognised staves to MusicXML would enable round-trip compatibility with notation software (MuseScore, Finale, Sibelius) and would require a post-processing pass that infers the missing notation-graphical structure from the LMX token stream and the original image.

References

- [1] E. Shatri and G. Fazekas, “OPTICAL MUSIC RECOGNITION: STATE OF THE ART AND MAJOR CHALLENGES”,
- [2] J. Calvo-Zaragoza, J. Hajič Jr., and A. Pacha, “Understanding Optical Music Recognition”, *ACM Computing Surveys*, vol. 53, no. 4, pp. 1–35, Jul. 2020, ISSN: 0360-0300. DOI: [10.1145/3397499](https://doi.org/10.1145/3397499) [Online]. Available: <https://dl.acm.org/doi/10.1145/3397499>
- [3] J. Hajič Jr., M. Kolárová, A. Pacha, and J. Calvo-Zaragoza, “How Current Optical Music Recognition Systems Are Becoming Useful for Digital Libraries”, pp. 57–61, 2018. DOI: [10.1145/3273024.3273034](https://doi.org/10.1145/3273024.3273034)
- [4] M. Reed, “Musical Form Reconstruction in Printed and Handwritten Lead Sheets via Optical Recognition of Chord Symbols”, Honors Thesis, University of Central Florida, Honors College, 2018. [Online]. Available: <https://stars.library.ucf.edu/honorsthesis/1462/>
- [5] J. C. Martínez-Sevilla, F. Foscarin, P. García-Iasci, D. Rizo, J. Calvo-Zaragoza, and G. Widmer, “Optical Music Recognition of Jazz Lead Sheets”, in *Proceedings of the 26th International Society for Music Information Retrieval Conference (ISMIR)*, 2025. [Online]. Available: <https://arxiv.org/abs/2509.05329>
- [6] “PrIMuS dataset”, Accessed: Feb. 18, 2026. [Online]. Available: <https://grfia.dlsi.ua.es/primus/>
- [7] L. Tuggener, Y. P. Satyawan, A. Pacha, J. Schmidhuber, and T. Stadelmann, *DeepScores V2*, Zenodo, Sep. 2, 2020. DOI: [10.5281/zenodo.4012193](https://doi.org/10.5281/zenodo.4012193) Accessed: Feb. 18, 2026. [Online]. Available: <https://zenodo.org/records/4012193>
- [8] *OMR-Research/lmx*, Optical Music Recognition Research, Nov. 24, 2025. Accessed: Feb. 23, 2026. [Online]. Available: <https://github.com/OMR-Research/lmx>
- [9] “Symbol Error Rate - an overview | ScienceDirect Topics”, Accessed: Feb. 18, 2026. [Online]. Available: <https://www.sciencedirect.com/topics/computer-science/symbol-error-rate>
- [10] A. Ríos-Vila, J. Calvo-Zaragoza, and T. Paquet, “Sheet Music Transformer: End-to-End Optical Music Recognition Beyond Monophonic Transcription”, in *Document Analysis and Recognition – ICDAR 2024*, ser. Lecture Notes in Computer Science, Springer, 2024, pp. 20–37. DOI: [10.1007/978-3-031-70552-6_2](https://doi.org/10.1007/978-3-031-70552-6_2) [Online]. Available: <https://arxiv.org/abs/2402.07596>
- [11] J. Mayer, M. Straka, J. Hajič Jr., and P. Pecina, “Practical End-to-End Optical Music Recognition for Pianoform Music”, in *Document Analysis and Recognition – ICDAR 2024*, ser. Lecture Notes in Computer Science, Springer, 2024, pp. 55–71. DOI: [10.1007/978-3-031-70552-6_4](https://doi.org/10.1007/978-3-031-70552-6_4) [Online]. Available: <https://arxiv.org/abs/2403.13763>
- [12] C. Dalitz, M. Droettboom, B. Pranzas, and I. Fujinaga, “A Comparative Study of Staff Removal Algorithms”, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 30, no. 5, pp. 753–766, May 2008, ISSN: 0162-8828. DOI: [10.1109/TPAMI.2007.70749](https://doi.org/10.1109/TPAMI.2007.70749) Accessed: Feb. 18, 2026. [Online]. Available: <http://ieeexplore.ieee.org/document/4359356/>

- [13] D. Bainbridge and T. Bell, “The challenge of optical music recognition”, *Computers and the Humanities*, vol. 35, no. 2, pp. 95–121, 2001. DOI: [10.1023/A:1002485918032](https://doi.org/10.1023/A:1002485918032)
- [14] A. Rebelo, I. Fujinaga, F. Paszkiewicz, A. R. S. Marcal, C. Guedes, and J. S. Cardoso, “Optical music recognition: State-of-the-art and open issues”, *International Journal of Multimedia Information Retrieval*, vol. 1, no. 3, pp. 173–190, 2012. DOI: [10.1007/s13735-012-0004-6](https://doi.org/10.1007/s13735-012-0004-6)
- [15] A. Fornés, A. Dutta, A. Gordo, and J. Lladós, “CVC-MUSCIMA: A ground truth of handwritten music score images for writer identification and staff removal”, *International Journal on Document Analysis and Recognition (IJDAR)*, vol. 15, no. 3, pp. 243–251, Sep. 2012, ISSN: 1433-2833, 1433-2825. DOI: [10.1007/s10032-011-0168-2](https://doi.org/10.1007/s10032-011-0168-2) Accessed: Feb. 18, 2026. [Online]. Available: <http://link.springer.com/10.1007/s10032-011-0168-2>
- [16] H. Bitteur, *Audiveris: Open-source Optical Music Recognition*. Accessed: May 18, 2026. [Online]. Available: <https://github.com/Audiveris/audiveris>
- [17] J. Calvo-Zaragoza and D. Rizo, “End-to-end neural optical music recognition of monophonic scores”, *Applied Sciences*, vol. 8, no. 4, p. 606, Apr. 16, 2018, ISSN: 2076-3417. DOI: [10.3390/app8040606](https://doi.org/10.3390/app8040606) [Online]. Available: <https://www.mdpi.com/2076-3417/8/4/606>
- [18] J. Calvo-Zaragoza and D. Rizo, “Camera-PrIMuS: Neural End-to-End Optical Music Recognition on Realistic Monophonic Scores”, in *Proceedings of the 19th International Society for Music Information Retrieval Conference (ISMIR)*, 2018, pp. 248–255. [Online]. Available: https://ismir2018.ircam.fr/doc/pdfs/33_Paper.pdf
- [19] A. Ríos-Vila, D. Rizo, J. Calvo-Zaragoza, and T. Paquet. “Sheet Music Transformer++: End-to-End Full-Page Optical Music Recognition for Pianoform Sheet Music”. arXiv: [2405.12105](https://arxiv.org/abs/2405.12105). [Online]. Available: <https://arxiv.org/abs/2405.12105>
- [20] G. Yang et al. “LEGATO: Large-scale End-to-end Generalizable Approach to Typeset OMR”. arXiv: [2506.19065](https://arxiv.org/abs/2506.19065). [Online]. Available: <https://arxiv.org/abs/2506.19065>
- [21] J. Hajič Jr. and P. Pecina, “The MUSCIMA++ Dataset for Handwritten Optical Music Recognition”, in *2017 14th IAPR International Conference on Document Analysis and Recognition (ICDAR)*, IEEE, 2017, pp. 39–46. DOI: [10.1109/ICDAR.2017.16](https://doi.org/10.1109/ICDAR.2017.16) [Online]. Available: <https://arxiv.org/abs/1703.04824>
- [22] E. Shatri and G. Fazekas. “DoReMi: First Glance at a Universal OMR Dataset”. arXiv: [2107.07786](https://arxiv.org/abs/2107.07786). [Online]. Available: <https://arxiv.org/abs/2107.07786>
- [23] J. C. Martínez-Sevilla, F. Foscari, D. Rizo, J. Calvo-Zaragoza, and G. Widmer. “Sheet Music Benchmark: Standardized Optical Music Recognition Evaluation”. arXiv: [2506.10488](https://arxiv.org/abs/2506.10488). [Online]. Available: <https://arxiv.org/abs/2506.10488>
- [24] Departament de Gestió, FIB–UPC, *Project management – module 2.4: Economic management*, GEP course material (internal). Covers budget structure (staff costs, general costs, amortisation, contingency, incidentals) and prescribes the x1.35 salary multiplier for Spanish employer social security contributions., Facultat d’Informàtica de Barcelona (FIB), Universitat Politècnica de Catalunya, 2025.
- [25] Instituto Nacional de Estadística (INE). “Encuesta anual de estructura salarial (EAES). año 2023. datos definitivos”. Official Spanish annual earnings survey (INEbase). The national mean gross annual salary in 2023 was 28,049.94 EUR. ICT-sector workers (NACE Section J) earn substantially above the mean, consistent with the 30,000–38,000 EUR gross range used here., Accessed: Mar. 9, 2026. [Online]. Available: https://www.ine.es/dyngs/INEbase/es/operacion.htm?c=Estadistica_C&cid=1254736177025&menu=ultiDatos&idp=1254735976596

- [26] Ministerio de Inclusión, Seguridad Social y Migraciones. “Cotización y recaudación de trabajadores. regímenes”. Official employer social security contribution rates for the Spanish General Regime. Total employer overhead is approximately 30 percent, commonly rounded to a x1.35 multiplier on gross salary in academic project budgets, as prescribed by the GEP economic management module., Accessed: Mar. 9, 2026. [Online]. Available: <https://www.seg-social.es/wps/portal/wss/internet/Trabajadores/CotizacionRecaudacionTrabajadores/10721>
- [27] Red Eléctrica de España (REE) / Redeia, “Informe del sistema eléctrico español 2023”, Red Eléctrica de España, Tech. Rep., 2024, Annual report of the Spanish electricity system. The CO2 emission intensity of the Spanish grid in 2023 was approximately 0.23 kg CO2/kWh, as also reported by MITECO. Accessed: Mar. 9, 2026. [Online]. Available: <https://www.ree.es/es/datos/publicaciones/informe-anual-sistema>
- [28] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*, 7th ed. Newtown Square, PA, USA: Project Management Institute, 2021, ISBN: 978-1-62825-664-2.
- [29] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, “Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks”, in *Proceedings of the 23rd International Conference on Machine Learning*, ACM, 2006, pp. 369–376. DOI: [10.1145/1143844.1143891](https://doi.org/10.1145/1143844.1143891)
- [30] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition”, in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778. DOI: [10.1109/CVPR.2016.90](https://doi.org/10.1109/CVPR.2016.90)
- [31] S. Hochreiter and J. Schmidhuber, “Long short-term memory”, *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. DOI: [10.1162/neco.1997.9.8.1735](https://doi.org/10.1162/neco.1997.9.8.1735)
- [32] “Albumentations: Fast and flexible image augmentations”, Accessed: Feb. 25, 2026. [Online]. Available: <https://albumentations.ai/>
- [33] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization”, in *International Conference on Learning Representations*, 2019. [Online]. Available: <https://openreview.net/forum?id=Bkg6RiCqY7>
- [34] L. N. Smith and N. Topin, “Super-convergence: Very fast training of neural networks using large learning rates”, in *Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications*, vol. 11006, SPIE, 2019, pp. 369–386. DOI: [10.1117/12.2520589](https://doi.org/10.1117/12.2520589)
- [35] Departament de Gestió, FIB–UPC, *Sustainability analysis for the bachelor’s thesis*, GEP course material (internal). Defines the sustainability matrix (Environmental, Economic, Social x PPP, Exploitation, Risks) and the questions for the initial and final milestones., Facultat d’Informàtica de Barcelona (FIB), Universitat Politècnica de Catalunya, 2018.
- [36] Ministerio para la Transición Ecológica y el Reto Demográfico (MITECO). “Factores de emisión: Electricidad”. MITECO publishes annual CO2 emission factors for the Spanish electricity mix. The 2023 factor is approximately 0.23 kg CO2/kWh., Accessed: Mar. 9, 2026. [Online]. Available: https://www.miteco.gob.es/content/dam/miteco/es/energia/files-1/Eficiencia/RITE/documentosreconocidosrite/Otros%20documentos/Factores_emision_CO2.pdf

Appendix A

Additional Tables and Hyperparameters

[TODO: Add `tab:hyperparams-full`—the complete hyperparameter grid search results for `img_height`, `batch_size`, and `BiLSTM` hidden size. Source: output of poetry run `python scripts/hyperparameter_search.py`.]

Appendix B

Qualitative Predictions

[TODO: Add fig:qual-worst-5—the five highest-SER predictions from the test set. Generate with: poetry run python src/cli.py evaluate -split test -export-worst 5 -output-dir figures/worst/. Include input strip, predicted tokens, ground-truth tokens, and edit operations for each.]

Appendix C

Code Snippets

[TODO: Include the three most instructive scripts: (1) the rendering loop from `src/data_processing/generate_realbook.py`, (2) the augmentation pipeline configuration from `src/data_processing/augment_scanned.py`, (3) the BNF grammar rule set from `src/omr_pipeline/grammar_fix.py`.]